



Adventist University of Central Africa

P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Department of Information Technology

Course Code and Name: INSY 8311 – Database Development with PL/SQL

Course Notes

Contents

Chapter 1: Introduction to PL/SQL

Chapter 2: Control Structures and Loops

Chapter 3: Cursors and Advanced Data Processing

Chapter 4: Procedures and Functions: Building Reusable Code

Chapter 5: Packages: Organizing Related Code

Chapter 6: Exception Handling: Managing Errors Gracefully

Chapter 7: Triggers: Automating Database Actions

Chapter 1: Introduction to PL/SQL

1.1 What is PL/SQL?

PL/SQL stands for **Procedural Language/Structured Query Language**. It is Oracle's programming language that extends SQL by adding programming features such as variables, conditions, and loops.

Understanding the Difference

SQL is used to ask questions and make changes to data:

```
-- Simple SQL commands
SELECT first_name, gpa FROM students;
UPDATE students SET gpa = 3.5 WHERE student_id = 1001;
DELETE FROM enrollments WHERE enrollment_id = 75;
```

PL/SQL lets you write complete programs with logic:

```
-- PL/SQL can make decisions
BEGIN
  IF v_gpa >= 3.5 THEN
    add_to_deans_list(v_student_id);
  END IF;
END;
```

Real-World Analogy

Think of managing student records like cooking.

SQL is like single cooking actions:

- ✓ "Get the eggs" (SELECT)
- ✓ "Add salt" (INSERT)
- ✓ "Remove the peel" (DELETE)

PL/SQL is like a complete recipe with instructions:

- ✓ "IF eggs are boiling THEN reduce heat"
- ✓ "WHILE sauce is thin, keep stirring"
- ✓ "FOR each guest, prepare one plate"

1.2 Why Do We Need PL/SQL?

Problem: Multiple Database Calls are Slow

Scenario: giving a 5% raise to all 40 staff members in the EMPLOYEES table.

Using only SQL (many round trips):

```
-- Step 1: Get employee data
SELECT employee_id, salary FROM employees;

-- Step 2: Your application calculates the new salary for each employee

-- Step 3: Send 40 separate UPDATE commands:
UPDATE employees SET salary = 89250 WHERE employee_id = 1;
UPDATE employees SET salary = 78750 WHERE employee_id = 2;
UPDATE employees SET salary = 68250 WHERE employee_id = 3;
-- ... (37 more times!)
```

Using PL/SQL (one call does everything):

```
BEGIN
  FOR emp IN (SELECT employee_id, salary FROM employees) LOOP
    UPDATE employees
      SET salary = emp.salary * 1.05
      WHERE employee_id = emp.employee_id;
  END LOOP;
  COMMIT;
END;
```

Result: the database updates all 40 employees in a single call, which is much faster.

Benefits:

- ✓ Faster execution (one call instead of many)
- ✓ Less network traffic
- ✓ Logic stays in the database (more secure)
- ✓ All applications use the same rules

1.3 Basic Structure of PL/SQL

The PL/SQL Block

Every PL/SQL program is made of blocks. A block has three parts.

Basic Syntax Structure

```
DECLARE
  -- Section 1: Declare variables here (OPTIONAL)
  -- This is where you create storage boxes for data
  variable_name datatype;
BEGIN
  -- Section 2: Executable statements here (REQUIRED)
  -- This is where the actual work happens
  -- SQL and PL/SQL statements go here
EXCEPTION
  -- Section 3: Handle errors here (OPTIONAL)
  -- What to do if something goes wrong
END;
/
```

Important Rules

- ✓ BEGIN and END are required. Every PL/SQL block must have them.
- ✓ DECLARE is optional. Use it only if you need variables.
- ✓ EXCEPTION is optional. Use it only if you want to handle errors.
- ✓ Every statement ends with a semicolon (;).
- ✓ The block ends with a forward slash (/) on a new line.

Your First PL/SQL Program

Example 1: Simple "Hello World" Program

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello from PL/SQL!');
END;
/
```

Explanation:

- BEGIN: start of the program

- DBMS_OUTPUT.PUT_LINE(): command to display text (like print in other languages)
- END; : end of the program
- / : execute the block

Using Variables to Store Information

Variables are like boxes where you store information temporarily.

Variable Declaration Syntax

```
DECLARE
    variable_name datatype;
    variable_name datatype := initial_value;
BEGIN
    -- use the variable here
END;
```

Example 2: Using Variables

```
DECLARE
    v_name          VARCHAR2(50);    -- Can store text up to 50 characters
    v_salary        NUMBER;          -- Can store numbers
    v_hire_date     DATE;            -- Can store dates
BEGIN
    -- Get data from the database and store it in variables
    SELECT first_name, salary, hire_date
    INTO v_name, v_salary, v_hire_date
    FROM employees
    WHERE employee_id = 1;

    -- Display the information
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary: $' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Hire Date: ' || v_hire_date);
END;
/
```

Output:

```
Employee Name: John
Salary: $85000
Hire Date: 15-JAN-18
```

Common Data Types for Beginners

Data Type	What it Stores	Example
VARCHAR2(size)	Text/characters	VARCHAR2(50) can store "Alice Johnson"
NUMBER	Numbers (whole or decimal)	NUMBER(3,2) can store a GPA such as 3.85
DATE	Dates and times	DATE can store DATE '2023-09-01'
BOOLEAN	True or False values	BOOLEAN can store TRUE or FALSE

Making Decisions with IF Statements

IF Statement Syntax

```
IF condition THEN
    -- do this if condition is true
ELSIF another_condition THEN
    -- do this if the second condition is true
ELSE
    -- do this if all conditions are false
END IF;
```

Example 3: Check a Student's Academic Standing

```
DECLARE
    v_name      VARCHAR2(50);
    v_gpa       NUMBER(3,2);
    v_standing  VARCHAR2(20);
BEGIN
    -- Get student information
    SELECT first_name, gpa
    INTO v_name, v_gpa
    FROM students
    WHERE student_id = 1001;

    -- Determine academic standing
    IF v_gpa >= 3.5 THEN
        v_standing := 'First Class';
    ELSIF v_gpa >= 3.0 THEN
        v_standing := 'Second Class';
    ELSE
        v_standing := 'Pass';
    END IF;

    -- Display result
    DBMS_OUTPUT.PUT_LINE(v_name || ' has ' || v_standing ||
        ' standing with a GPA of ' || v_gpa);
END;
/
```

Output:

```
Alice has First Class standing with a GPA of 3.8
```

1.4 Working with the Lab Tables: Practical Examples

The Lab Database

All examples in these notes use the tables created in the lab setup script (run it while connected as the lecturer user in SQL Developer). The script creates five tables:

Table	Main Columns	Sample Data
STUDENTS	student_id, first_name, last_name, email, phone, date_of_birth, enrollment_date, gpa, status	50 students, IDs 1001 to 1050
COURSES	course_id, course_name, course_code, credits, instructor, department, max_students, semester	15 courses, IDs 101 to 115
ENROLLMENTS	enrollment_id, student_id, course_id, enrollment_date, grade, score, status	75 enrollments linking students to courses

Table	Main Columns	Sample Data
EMPLOYEES	employee_id, first_name, last_name, email, phone, hire_date, job_title, salary, commission_pct, department, manager_id	40 staff members, IDs 1 to 40
DEPARTMENTS	department_id, department_name, manager_id, location, budget	6 departments, IDs 10 to 60

Points to keep in mind:

- ✓ ENROLLMENTS.student_id refers to STUDENTS, and ENROLLMENTS.course_id refers to COURSES (foreign keys).
- ✓ A student can be enrolled in a course only once: UNIQUE(student_id, course_id).
- ✓ STUDENTS.gpa must be between 0 and 4, and STUDENTS.email must be unique.
- ✓ EMPLOYEES.department holds the department **name** (for example 'Computer Science'), so it is joined to DEPARTMENTS through department_name.
- ✓ EMPLOYEES.manager_id refers to another employee, and salaries are annual amounts.
- ✓ Some enrollments have no grade or score yet (NULL), because the course is still in progress.

Example 4: Calculate Years of Service

```

DECLARE
    v_name          VARCHAR2(100);
    v_hire_date     DATE;
    v_years         NUMBER;
BEGIN
    -- Get employee info
    SELECT first_name || ' ' || last_name, hire_date
    INTO v_name, v_hire_date
    FROM employees
    WHERE employee_id = 1;

    -- Calculate years (MONTHS_BETWEEN gives months, divide by 12 for years)
    v_years := ROUND(MONTHS_BETWEEN(SYSDATE, v_hire_date) / 12, 1);

    -- Display result
    DBMS_OUTPUT.PUT_LINE(v_name || ' has worked for ' || v_years || ' years');
END;
/

```

Explanation:

- ✓ SYSDATE: the current date
- ✓ MONTHS_BETWEEN: calculates the number of months between two dates
- ✓ ROUND: rounds the number to 1 decimal place
- ✓ || : concatenates (joins) text together

Example 5: Give a Raise to an Employee

```

DECLARE
    v_emp_id        NUMBER := 5;      -- Employee ID we want to update
    v_current_salary NUMBER;
    v_new_salary     NUMBER;
    v_increase_percent NUMBER := 10;  -- 10% raise
BEGIN
    -- Get current salary
    SELECT salary INTO v_current_salary
    FROM employees
    WHERE employee_id = v_emp_id;

```

```

-- Calculate new salary
v_new_salary := v_current_salary +
                (v_current_salary * v_increase_percent / 100);

-- Update the salary
UPDATE employees
SET salary = v_new_salary
WHERE employee_id = v_emp_id;

-- Save the change
COMMIT;

-- Display what we did
DBMS_OUTPUT.PUT_LINE('Salary increased from $' || v_current_salary ||
                    ' to $' || v_new_salary);
END;
/

```

Key concepts:

- ✓ := is used to assign values to variables
- ✓ COMMIT saves the changes to the database
- ✓ You can do calculations with variables just like in math

Example 6: Check a Department's Salary Cost Against its Budget

```

DECLARE
    v_dept_id      NUMBER := 10;    -- Computer Science
    v_dept_name    VARCHAR2(50);
    v_budget       NUMBER;
    v_total_salary NUMBER;
    v_emp_count    NUMBER;
BEGIN
    -- Get the department name and budget
    SELECT department_name, budget
    INTO v_dept_name, v_budget
    FROM departments
    WHERE department_id = v_dept_id;

    -- Calculate the total salary for the department
    SELECT COUNT(*), NVL(SUM(salary), 0)
    INTO v_emp_count, v_total_salary
    FROM employees
    WHERE department = v_dept_name;

    -- Display information
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept_name);
    DBMS_OUTPUT.PUT_LINE('Number of Employees: ' || v_emp_count);
    DBMS_OUTPUT.PUT_LINE('Total Salary Cost: $' || v_total_salary);
    DBMS_OUTPUT.PUT_LINE('Budget: $' || v_budget);

    -- Check whether the department is over budget
    IF v_total_salary > v_budget THEN
        DBMS_OUTPUT.PUT_LINE('WARNING: Over budget by $' ||
                            (v_total_salary - v_budget));
    ELSE
        DBMS_OUTPUT.PUT_LINE('Within budget. Remaining: $' ||
                            (v_budget - v_total_salary));
    END IF;
END;
/

```

Output (with the original lab data):

```
Department: Computer Science
Number of Employees: 16
Total Salary Cost: $1147000
Budget: $750000
WARNING: Over budget by $397000
```

New functions used:

- ✓ COUNT(*): counts the number of rows
- ✓ SUM(salary): adds up all salaries
- ✓ NVL(value, 0): if the value is NULL, use 0 instead

1.5 Handling Errors (Basic Exception Handling)

Sometimes things go wrong. PL/SQL lets you handle errors gracefully.

Basic Exception Handling Syntax

```
BEGIN
    -- your code here
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- handle when no rows are found
    WHEN OTHERS THEN
        -- handle all other errors
END;
```

Example 7: Safe Student Lookup

```
DECLARE
    v_name          VARCHAR2(100);
    v_student_id NUMBER := 9999;    -- This student doesn't exist!
BEGIN
    -- Try to get the student name
    SELECT first_name || ' ' || last_name INTO v_name
    FROM students
    WHERE student_id = v_student_id;

    DBMS_OUTPUT.PUT_LINE('Student found: ' || v_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Student ID ' || v_student_id || ' not found!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred');
END;
/
```

Output:

```
Error: Student ID 9999 not found!
```

Without exception handling, the program would stop with an error.

1.6 Important PL/SQL Commands for Beginners

Command	What it Does	Example
DBMS_OUTPUT.PUT_LINE()	Display text on screen	DBMS_OUTPUT.PUT_LINE('Hello');

Command	What it Does	Example
SELECT ... INTO	Get data from the database into variables	SELECT gpa INTO v_gpa FROM students ...
:=	Assign a value to a variable	v_total := 100;
	Join text together	'Hello' ' ' 'World'
COMMIT	Save changes to the database	COMMIT;
ROLLBACK	Undo changes	ROLLBACK;

Before Running PL/SQL Code

In SQL*Plus or SQL Developer, always run this first so that you can see the output:

```
SET SERVEROUTPUT ON;
```

1.7 Step-by-Step: Writing Your First Program

Complete Example: Employee Bonus Calculator

Business rule: give a 15% bonus if the salary is below \$60,000, otherwise 10%.

```
-- Step 1: Enable output
SET SERVEROUTPUT ON;

-- Step 2: Write the PL/SQL block
DECLARE
    -- Declare variables
    v_emp_id          NUMBER := 5;
    v_emp_name        VARCHAR2(100);
    v_salary           NUMBER;
    v_bonus            NUMBER;
    v_bonus_percent    NUMBER;
BEGIN
    -- Get employee information
    SELECT first_name || ' ' || last_name, salary
    INTO v_emp_name, v_salary
    FROM employees
    WHERE employee_id = v_emp_id;

    -- Determine bonus percentage
    IF v_salary < 60000 THEN
        v_bonus_percent := 15;
    ELSE
        v_bonus_percent := 10;
    END IF;

    -- Calculate bonus amount
    v_bonus := v_salary * v_bonus_percent / 100;

    -- Update employee record
    UPDATE employees
    SET salary = salary + v_bonus
    WHERE employee_id = v_emp_id;

    -- Save changes
    COMMIT;

    -- Display results
    DBMS_OUTPUT.PUT_LINE('==== BONUS CALCULATION =====');
    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp_name);
    DBMS_OUTPUT.PUT_LINE('Original Salary: $' || v_salary);
```

```

DBMS_OUTPUT.PUT_LINE('Bonus Percent: ' || v_bonus_percent || '%');
DBMS_OUTPUT.PUT_LINE('Bonus Amount: $' || v_bonus);
DBMS_OUTPUT.PUT_LINE('New Salary: $' || (v_salary + v_bonus));
DBMS_OUTPUT.PUT_LINE('=====');

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee not found!');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    ROLLBACK;    -- Undo changes if there is an error
END;
/

```

Summary of Chapter 1

1. What PL/SQL is: a programming language that extends SQL with procedural features.
2. Why we use it: to reduce network traffic and keep business logic in the database.
3. Block structure: DECLARE (optional), BEGIN (required), EXCEPTION (optional), END.
4. Variables: how to declare and use them to store data.
5. Data types: VARCHAR2, NUMBER, DATE, BOOLEAN.
6. IF statements: how to make decisions in your code.
7. SELECT INTO: how to get data from tables into variables.
8. Exception handling: how to handle errors gracefully.

Practice Exercises

Use the lab tables for these exercises:

1. Basic Output: Write a PL/SQL block that displays "Welcome to PL/SQL Programming!"
2. Student Info: Write a block that gets and displays the full name and email of student 1005.
3. GPA Check: Create a block that checks whether student 1010's GPA is 3.0 or above. Display an appropriate message.
4. Course Count: Write a block that counts how many students are enrolled in course 103 (PL/SQL Programming) and displays the result.
5. Salary Update: Give a 20% raise to employee 10. Display the old and new salary.
6. Error Handling: Try to get information for course_id 999. Handle the error and display a friendly message.

Key Syntax to Remember

```
-- Basic block structure
DECLARE
    variable_name datatype := value;
BEGIN
    SELECT column INTO variable FROM table WHERE condition;

    IF condition THEN
        statements;
    END IF;

    COMMIT;
EXCEPTION
    WHEN exception_name THEN
        error_handling;
END;
/

-- Always enable output first
SET SERVEROUTPUT ON;
```

Chapter 2: Control Structures and Loops

In this chapter you will learn:

- ✓ How to make complex decisions in your programs
- ✓ How to repeat actions multiple times (loops)
- ✓ How to process many students, courses or employees efficiently
- ✓ How to control the flow of your PL/SQL programs

2.1 Advanced Decision Making with IF Statements

In Chapter 1 we learned basic IF statements. Now let us look at more complex decision-making scenarios.

Nested IF Statements

Sometimes you need to check one condition inside another.

Nested IF Syntax

```
IF condition1 THEN
  IF condition2 THEN
    -- do this if both conditions are true
  ELSE
    -- do this if condition1 is true but condition2 is false
  END IF;
ELSE
  -- do this if condition1 is false
END IF;
```

Example 1: Employee Promotion Eligibility

Business rule: an employee can be promoted if they have worked for more than 3 years AND their salary is below \$90,000.

```
DECLARE
  v_emp_id      NUMBER := 5;
  v_name        VARCHAR2(100);
  v_hire_date   DATE;
  v_salary      NUMBER;
  v_years_worked NUMBER;
BEGIN
  -- Get employee information
  SELECT first_name || ' ' || last_name, hire_date, salary
  INTO v_name, v_hire_date, v_salary
  FROM employees
  WHERE employee_id = v_emp_id;

  -- Calculate years worked
  v_years_worked := ROUND(MONTHS_BETWEEN(SYSDATE, v_hire_date) / 12, 1);

  -- Check promotion eligibility
  IF v_years_worked > 3 THEN
    IF v_salary < 90000 THEN
      DBMS_OUTPUT.PUT_LINE(v_name || ' is ELIGIBLE for promotion');
      DBMS_OUTPUT.PUT_LINE('Years worked: ' || v_years_worked);
      DBMS_OUTPUT.PUT_LINE('Current salary: $' || v_salary);
    ELSE
      DBMS_OUTPUT.PUT_LINE(v_name ||
        ' has enough experience but salary too high');
    END IF;
  ELSE
    DBMS_OUTPUT.PUT_LINE(v_name || ' needs more experience (currently ' ||
      v_years_worked || ' years)');
  END IF;
END;
```

```

        END IF;
END;
/

```

Logical Operators: AND, OR, NOT

You can combine multiple conditions using logical operators.

Operator	Meaning	Example
AND	Both conditions must be true	IF gpa >= 3.5 AND status = 'Active'
OR	At least one condition must be true	IF department = 'Business' OR department = 'Mathematics'
NOT	Reverses the condition	IF NOT (score >= 50)

Example 2: Bonus Eligibility with Multiple Conditions

```

DECLARE
    v_emp_id          NUMBER := 1;
    v_name            VARCHAR2(100);
    v_salary          NUMBER;
    v_department      VARCHAR2(50);
    v_commission_pct  NUMBER;
BEGIN
    SELECT first_name || ' ' || last_name, salary, department, commission_pct
    INTO v_name, v_salary, v_department, v_commission_pct
    FROM employees
    WHERE employee_id = v_emp_id;

    -- Employee gets a bonus if:
    -- (Salary < 70000 AND in the Business department) OR has a commission
    IF (v_salary < 70000 AND v_department = 'Business')
        OR v_commission_pct IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE(v_name || ' qualifies for bonus!');
        DBMS_OUTPUT.PUT_LINE('Salary: $' || v_salary);
        DBMS_OUTPUT.PUT_LINE('Department: ' || v_department);
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_name || ' does not qualify for bonus');
    END IF;
END;
/

```

2.2 CASE Statements: Cleaner Decision Making

When you have many conditions to check, CASE statements are cleaner than a long chain of IF-ELSIF statements.

Simple CASE Syntax

```

CASE variable
    WHEN value1 THEN
        -- do this if variable = value1
    WHEN value2 THEN
        -- do this if variable = value2
    ELSE
        -- do this if there is no match
END CASE;

```

Searched CASE Syntax

```
CASE
  WHEN condition1 THEN
    -- do this if condition1 is true
  WHEN condition2 THEN
    -- do this if condition2 is true
  ELSE
    -- do this if no condition is true
END CASE;
```

Example 3: Department Number to Full Description

```
DECLARE
  v_dept_id    NUMBER := 40;
  v_dept_desc  VARCHAR2(100);
BEGIN
  -- Simple CASE: match exact values
  v_dept_desc := CASE v_dept_id
    WHEN 10 THEN 'Computer Science - Building A'
    WHEN 20 THEN 'Mathematics - Building B'
    WHEN 30 THEN 'Business - Building C'
    WHEN 40 THEN 'Engineering - Building D'
    WHEN 50 THEN 'Psychology - Building E'
    WHEN 60 THEN 'English - Building F'
    ELSE 'Unknown Department'
  END;

  DBMS_OUTPUT.PUT_LINE('Department ' || v_dept_id || ': ' || v_dept_desc);
END;
/
```

Example 4: Letter Grade from a Score

```
DECLARE
  v_enrollment_id NUMBER := 1;
  v_student_name   VARCHAR2(100);
  v_course_name    VARCHAR2(100);
  v_score          NUMBER;
  v_letter         VARCHAR2(2);
  v_remark         VARCHAR2(30);
BEGIN
  SELECT s.first_name || ' ' || s.last_name, c.course_name, e.score
  INTO v_student_name, v_course_name, v_score
  FROM enrollments e
  JOIN students s ON e.student_id = s.student_id
  JOIN courses c  ON e.course_id = c.course_id
  WHERE e.enrollment_id = v_enrollment_id;

  -- Searched CASE: check conditions
  CASE
    WHEN v_score >= 90 THEN
      v_letter := 'A';
      v_remark := 'Excellent';
    WHEN v_score >= 80 THEN
      v_letter := 'B';
      v_remark := 'Very Good';
    WHEN v_score >= 70 THEN
      v_letter := 'C';
      v_remark := 'Good';
    WHEN v_score >= 50 THEN
      v_letter := 'D';
      v_remark := 'Pass';
    ELSE
      v_letter := 'F';
      v_remark := 'Fail';
  END CASE;

  DBMS_OUTPUT.PUT_LINE('Student: ' || v_student_name);
  DBMS_OUTPUT.PUT_LINE('Course: ' || v_course_name);
```

```

        DBMS_OUTPUT.PUT_LINE('Score: ' || v_score);
        DBMS_OUTPUT.PUT_LINE('Grade: ' || v_letter || ' (' || v_remark || ')');
END;
/

```

Output:

```

Student: Alice Johnson
Course: Introduction to Programming
Score: 92.5
Grade: A (Excellent)

```

2.3 Introduction to Loops

Loops allow you to repeat actions multiple times. This is essential when working with many records.

Why Use Loops?

Without loops, updating 50 students one by one would require writing 50 UPDATE statements. With loops, you write the logic once and the loop goes through all the rows automatically.

Simple LOOP: The Basic Loop

Simple LOOP Syntax

```

LOOP
    -- statements to repeat
    EXIT WHEN condition;    -- exit the loop when this is true
END LOOP;

```

Example 5: Count from 1 to 5

```

DECLARE
    v_counter NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE('Count: ' || v_counter);
        v_counter := v_counter + 1;

        EXIT WHEN v_counter > 5;    -- Stop when counter exceeds 5
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Loop finished!');
END;
/

```

Output:

```

Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Loop finished!

```

Warning: always make sure your loop has an EXIT condition. Otherwise it will run forever (an infinite loop) and you will have to stop it manually.

WHILE LOOP: Loop While a Condition is True

WHILE LOOP Syntax

```
WHILE condition LOOP
    -- statements to repeat
    -- make sure the condition eventually changes!
END LOOP;
```

Example 6: Years for a Department Budget to Reach a Target

The Computer Science budget is 750,000. How many years will it take to reach 1,000,000 if it grows by 5% per year?

```
DECLARE
    v_budget NUMBER;
    v_target NUMBER := 1000000;
    v_rate  NUMBER := 0.05;  -- 5% growth per year
    v_year  NUMBER := 0;
BEGIN
    SELECT budget INTO v_budget
    FROM departments
    WHERE department_id = 10;

    DBMS_OUTPUT.PUT_LINE('Starting Budget: $' || v_budget);
    DBMS_OUTPUT.PUT_LINE('Growth Rate: ' || (v_rate * 100) || '%');
    DBMS_OUTPUT.PUT_LINE('-----');

    WHILE v_budget < v_target LOOP  -- Continue until the target is reached
        v_year := v_year + 1;
        v_budget := v_budget * (1 + v_rate);

        DBMS_OUTPUT.PUT_LINE('Year ' || v_year || ': $' || ROUND(v_budget, 2));
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('-----');
    DBMS_OUTPUT.PUT_LINE('Target reached after ' || v_year || ' years');
END;
/
```

FOR LOOP: Loop a Specific Number of Times

The FOR loop is the most commonly used loop when you know exactly how many times you want to repeat something.

FOR LOOP Syntax

```
FOR counter IN start_value..end_value LOOP
    -- statements to repeat
    -- counter automatically increases by 1
END LOOP;
```

Reverse FOR LOOP

```
FOR counter IN REVERSE start_value..end_value LOOP
    -- counter automatically decreases by 1
END LOOP;
```

Example 7: Multiplication Table

```
DECLARE
    v_number NUMBER := 5;
    v_result NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Multiplication Table for ' || v_number);
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR i IN 1..10 LOOP
        v_result := v_number * i;
        DBMS_OUTPUT.PUT_LINE(v_number || ' x ' || i || ' = ' || v_result);
    END LOOP;
END;
/
```

Output:

```
Multiplication Table for 5
=====
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Example 8: Countdown (Reverse Loop)

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Exam starts in:');

    FOR i IN REVERSE 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(i || '...');
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('You may begin!');
END;
/
```

2.4 Loop Control Statements

EXIT Statement

The EXIT statement lets you break out of a loop early.

Example 9: Find the First Employee Earning More than \$90,000

```
DECLARE
    v_name    VARCHAR2(100);
    v_salary  NUMBER;
    v_found   BOOLEAN := FALSE;
BEGIN
    FOR i IN 1..40 LOOP
        SELECT first_name || ' ' || last_name, salary
        INTO v_name, v_salary
        FROM employees
        WHERE employee_id = i;

        IF v_salary > 90000 THEN
            DBMS_OUTPUT.PUT_LINE('Found high earner: ' || v_name ||
                                ' (Salary: $' || v_salary || ')');
            v_found := TRUE;
            EXIT;    -- Stop searching, we found one!
        END IF;
    END LOOP;

    IF NOT v_found THEN
        DBMS_OUTPUT.PUT_LINE('No high earner found in this range');
    END IF;
END;
/
```

Output:

```
Found high earner: David Professor (Salary: $95000)
```

CONTINUE Statement

CONTINUE skips the rest of the current iteration and moves on to the next one.

Example 10: List Only the High-GPA Students

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Students 1001-1015 with a GPA of 3.5 or above:');
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR v_id IN 1001..1015 LOOP
        DECLARE
            v_name VARCHAR2(100);
            v_gpa  NUMBER(3,2);
        BEGIN
            SELECT first_name || ' ' || last_name, gpa
            INTO v_name, v_gpa
            FROM students
            WHERE student_id = v_id;

            -- Skip students below 3.5
            IF v_gpa < 3.5 THEN
                CONTINUE;    -- Skip to the next iteration
            END IF;

            -- This only executes for high-GPA students
            DBMS_OUTPUT.PUT_LINE(v_name || ': ' || v_gpa);
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                NULL;    -- Skip if the student doesn't exist
        END;
    END LOOP;
END;
/
```

2.5 Nested Loops: Loops Inside Loops

You can put loops inside other loops for more complex processing.

Example 11: Department and Staff Report

```
DECLARE
    v_emp_count NUMBER;
BEGIN
    -- Outer loop: go through departments
    FOR dept IN (SELECT department_id, department_name
                FROM departments
                ORDER BY department_id) LOOP

        DBMS_OUTPUT.PUT_LINE('');
        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('Department: ' || dept.department_name);
        DBMS_OUTPUT.PUT_LINE('=====');

        v_emp_count := 0;

        -- Inner loop: go through employees in this department
        FOR emp IN (SELECT first_name || ' ' || last_name AS name, salary
                    FROM employees
                    WHERE department = dept.department_name
                    ORDER BY salary DESC) LOOP

            v_emp_count := v_emp_count + 1;
            DBMS_OUTPUT.PUT_LINE(' ' || emp.name || ' - $' || emp.salary);
        END LOOP;

        IF v_emp_count = 0 THEN
            DBMS_OUTPUT.PUT_LINE(' No employees in this department');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Total employees: ' || v_emp_count);
        END IF;
    END LOOP;
END;
/
```

With the lab data, the English department (60) has no staff, so it prints "No employees in this department".

2.6 Processing Multiple Rows: Introduction to the Cursor FOR Loop

The cursor FOR loop is a special type of loop designed for processing database records. It is the easiest way to work with multiple rows.

Cursor FOR Loop Syntax

```
FOR record_name IN (SELECT query) LOOP
    -- process each row
    -- access columns using record_name.column_name
END LOOP;
```

Why use a cursor FOR loop?

- ✓ It automatically fetches each row one by one
- ✓ There is no need to declare variables for the columns
- ✓ It closes automatically when done
- ✓ The code is clean and easy to read

Example 12: Give Raises to All Employees

```
DECLARE
    v_raise_amount NUMBER;
    v_new_salary    NUMBER;
    v_count         NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Processing Annual Raises...');
    DBMS_OUTPUT.PUT_LINE('=====');

    -- Loop through all employees
    FOR emp IN (SELECT employee_id, first_name, last_name, salary
                FROM employees
                ORDER BY employee_id) LOOP

        -- Calculate 8% raise
        v_raise_amount := emp.salary * 0.08;
        v_new_salary := emp.salary + v_raise_amount;

        -- Update the salary
        UPDATE employees
        SET salary = v_new_salary
        WHERE employee_id = emp.employee_id;

        -- Display what we did
        DBMS_OUTPUT.PUT_LINE(emp.first_name || ' ' || emp.last_name ||
                              ': $' || emp.salary || ' -> $' ||
                              ROUND(v_new_salary, 2));

        v_count := v_count + 1;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Total employees updated: ' || v_count);
END;
/
```

Example 13: Conditional Processing in a Loop

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Performance-Based Bonus Distribution');
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR emp IN (SELECT employee_id, first_name, last_name, salary,
                    department, hire_date
                FROM employees) LOOP
        DECLARE
            v_bonus NUMBER := 0;
            v_years NUMBER;
        BEGIN
            v_years := ROUND(MONTHS_BETWEEN(SYSDATE, emp.hire_date) / 12, 1);

            -- Different bonus rules
            IF emp.department = 'Business' THEN -- Business department
                v_bonus := emp.salary * 0.15;
            ELSIF v_years > 5 THEN -- Long-serving staff
                v_bonus := emp.salary * 0.12;
            ELSIF emp.salary < 55000 THEN -- Lower-paid staff
                v_bonus := emp.salary * 0.10;
            ELSE
                v_bonus := emp.salary * 0.05; -- Everyone else
            END IF;

            -- Update salary with bonus
            UPDATE employees
            SET salary = salary + v_bonus
            WHERE employee_id = emp.employee_id;
        END;
    END LOOP;
END;
```

```

        DBMS_OUTPUT.PUT_LINE(emp.first_name || ' ' || emp.last_name ||
                               ' receives bonus: $' || ROUND(v_bonus, 2));
    END;
END LOOP;

COMMIT;
DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('All bonuses processed!');
END;
/

```

2.7 NULL Handling in Conditions

NULL is a special value meaning "unknown" or "no value". It requires special handling.

Important NULL rules:

- ✓ NULL is not equal to anything (not even another NULL)
- ✓ NULL is not "not equal" to anything either
- ✓ Use IS NULL or IS NOT NULL to check for NULL
- ✓ Use NVL(value, replacement) to provide default values

Example 14: Handling Enrollments That Have Not Been Graded

In the lab data, some enrollments have no score or grade yet. This block shows every course taken by student 1001.

```

BEGIN
    FOR enr IN (SELECT c.course_name, e.score, e.grade
                FROM enrollments e
                JOIN courses c ON e.course_id = c.course_id
                WHERE e.student_id = 1001
                ORDER BY c.course_id) LOOP

        DBMS_OUTPUT.PUT_LINE('Course: ' || enr.course_name);

        -- Check whether a score exists
        IF enr.score IS NULL THEN
            DBMS_OUTPUT.PUT_LINE('Result: Not yet graded');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Result: ' || enr.score || ' (' || enr.grade || ')');
        END IF;

        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
END;
/

```

Output:

```

Course: Introduction to Programming
Result: 92.5 (A)
-----
Course: Database Management Systems
Result: 88 (A)
-----
Course: PL/SQL Programming
Result: Not yet graded
-----
Course: Calculus I
Result: 85 (B)
-----

```

Example 15: Using NVL for Default Values

```
BEGIN
  FOR emp IN (SELECT first_name, last_name, salary, commission_pct, manager_id
              FROM employees
              WHERE employee_id <= 6) LOOP

    DBMS_OUTPUT.PUT_LINE('Employee: ' || emp.first_name || ' ' || emp.last_name);

    -- NVL provides a default value if the column is NULL
    DBMS_OUTPUT.PUT_LINE('Total Pay: $' ||
      (emp.salary + (emp.salary * NVL(emp.commission_pct, 0))));

    DBMS_OUTPUT.PUT_LINE('Manager ID: ' ||
      NVL(TO_CHAR(emp.manager_id), 'No Manager'));
    DBMS_OUTPUT.PUT_LINE('-----');
  END LOOP;
END;
```

2.8 Practical Applications: Real Scenarios

Example 16: Complete Salary Review System

```
DECLARE
  v_total_before NUMBER := 0;
  v_total_after  NUMBER := 0;
  v_emp_count    NUMBER;
BEGIN
  DBMS_OUTPUT.PUT_LINE('=====');
  DBMS_OUTPUT.PUT_LINE('          ANNUAL SALARY REVIEW SYSTEM');
  DBMS_OUTPUT.PUT_LINE('=====');

  -- Process each department
  FOR dept IN (SELECT department_id, department_name
              FROM departments
              ORDER BY department_id) LOOP

    v_emp_count := 0;
    v_total_before := 0;
    v_total_after := 0;

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('Department: ' || dept.department_name);
    DBMS_OUTPUT.PUT_LINE('-----');

    -- Process each employee in the department
    FOR emp IN (SELECT employee_id, first_name, last_name, salary, hire_date
                FROM employees
                WHERE department = dept.department_name
                ORDER BY salary DESC) LOOP

      DECLARE
        v_years      NUMBER;
        v_raise_pct  NUMBER;
        v_new_salary NUMBER;
      BEGIN
        v_years := ROUND(MONTHS_BETWEEN(SYSDATE, emp.hire_date) / 12, 1);
        v_emp_count := v_emp_count + 1;
        v_total_before := v_total_before + emp.salary;

        -- Determine raise percentage
        CASE
          WHEN v_years > 10 THEN v_raise_pct := 15;
          WHEN v_years > 5  THEN v_raise_pct := 12;
          WHEN v_years > 2  THEN v_raise_pct := 10;
          ELSE v_raise_pct := 8;
        END CASE;
```

```

-- Calculate new salary
v_new_salary := emp.salary * (1 + v_raise_pct / 100);
v_total_after := v_total_after + v_new_salary;

-- Update database
UPDATE employees
SET salary = v_new_salary
WHERE employee_id = emp.employee_id;

-- Display details
DBMS_OUTPUT.PUT_LINE(' ' || emp.first_name || ' ' || emp.last_name ||
' (' || v_years || ' yrs): $' || emp.salary ||
' -> $' || ROUND(v_new_salary, 2) ||
' (+ ' || v_raise_pct || '%)');
END;
END LOOP;

-- Department summary
IF v_emp_count > 0 THEN
  DBMS_OUTPUT.PUT_LINE(' -----');
  DBMS_OUTPUT.PUT_LINE(' Employees: ' || v_emp_count);
  DBMS_OUTPUT.PUT_LINE(' Total Before: $' || ROUND(v_total_before, 2));
  DBMS_OUTPUT.PUT_LINE(' Total After: $' || ROUND(v_total_after, 2));
  DBMS_OUTPUT.PUT_LINE(' Increase: $' ||
ROUND(v_total_after - v_total_before, 2));
ELSE
  DBMS_OUTPUT.PUT_LINE(' No employees in this department');
END IF;
END LOOP;

COMMIT;
DBMS_OUTPUT.PUT_LINE('');
DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('Salary review completed successfully!');
DBMS_OUTPUT.PUT_LINE('=====');
END;
/

```

Example 17: Students at Academic Risk

A student is flagged if their average score is well below the average of the courses they take, or if their GPA is low.

```

DECLARE
  v_risk_level          VARCHAR2(20);
  v_high_risk_count     NUMBER := 0;
  v_medium_risk_count   NUMBER := 0;
  v_low_risk_count      NUMBER := 0;
BEGIN
  DBMS_OUTPUT.PUT_LINE('STUDENT ACADEMIC RISK ANALYSIS');
  DBMS_OUTPUT.PUT_LINE('=====');

  FOR stu IN (SELECT student_id, first_name, last_name, gpa
              FROM students
              WHERE status = 'Active') LOOP
    DECLARE
      v_avg_score NUMBER;
    BEGIN
      -- Average of the student's graded courses
      SELECT AVG(score) INTO v_avg_score
      FROM enrollments
      WHERE student_id = stu.student_id
      AND score IS NOT NULL;

      -- Determine risk level
      IF stu.gpa < 3.0 AND NVL(v_avg_score, 0) < 75 THEN
        v_risk_level := 'HIGH RISK';
        v_high_risk_count := v_high_risk_count + 1;
      END IF;
    END;
  END LOOP;
END;

```

```

ELSIF stu.gpa < 3.0 OR NVL(v_avg_score, 100) < 75 THEN
    v_risk_level := 'MEDIUM RISK';
    v_medium_risk_count := v_medium_risk_count + 1;
ELSE
    v_risk_level := 'LOW RISK';
    v_low_risk_count := v_low_risk_count + 1;
END IF;

-- Display only high and medium risk students
IF v_risk_level IN ('HIGH RISK', 'MEDIUM RISK') THEN
    DBMS_OUTPUT.PUT_LINE(stu.first_name || ' ' || stu.last_name);
    DBMS_OUTPUT.PUT_LINE(' Status: ' || v_risk_level);
    DBMS_OUTPUT.PUT_LINE(' GPA: ' || stu.gpa);
    DBMS_OUTPUT.PUT_LINE(' Average score: ' ||
        NVL(TO_CHAR(ROUND(v_avg_score, 1)), 'No graded courses'));
    DBMS_OUTPUT.PUT_LINE(' ---');
END IF;
END;
END LOOP;

DBMS_OUTPUT.PUT_LINE('');
DBMS_OUTPUT.PUT_LINE('SUMMARY:');
DBMS_OUTPUT.PUT_LINE('High Risk: ' || v_high_risk_count || ' students');
DBMS_OUTPUT.PUT_LINE('Medium Risk: ' || v_medium_risk_count || ' students');
DBMS_OUTPUT.PUT_LINE('Low Risk: ' || v_low_risk_count || ' students');
END;
/

```

Summary of Chapter 2

1. Advanced IF statements: nested conditions and logical operators (AND, OR, NOT).
2. CASE statements: a cleaner way to handle multiple conditions.
3. Three types of loops: the simple LOOP with EXIT WHEN, the WHILE LOOP for condition-based repetition, and the FOR LOOP for counting and ranges.
4. Loop control: EXIT to break out, CONTINUE to skip an iteration.
5. Nested loops: loops inside loops for more complex processing.
6. Cursor FOR loop: the easiest way to process multiple database rows.
7. NULL handling: IS NULL, IS NOT NULL, and the NVL function.

Key Syntax Reference

```

-- IF with logical operators
IF condition1 AND condition2 THEN
    statements;
ELSIF condition3 OR condition4 THEN
    statements;
ELSE
    statements;
END IF;

-- CASE statement
CASE
    WHEN condition1 THEN statements;
    WHEN condition2 THEN statements;
    ELSE statements;
END CASE;

-- Simple LOOP
LOOP
    statements;
    EXIT WHEN condition;
END LOOP;

```

```

-- WHILE LOOP
WHILE condition LOOP
    statements;
END LOOP;

-- FOR LOOP
FOR i IN 1..10 LOOP
    statements;
END LOOP;

-- Cursor FOR LOOP
FOR record IN (SELECT columns FROM table WHERE condition) LOOP
    -- use record.column_name
    statements;
END LOOP;

-- NULL handling
IF variable IS NULL THEN
    statements;
END IF;

variable := NVL(nullable_value, default_value);

```

Practice Exercises

1. Nested IF: Write a program that checks whether student 1005 qualifies for a scholarship. Requirements: GPA of 3.8 or above AND status 'Active'.
2. CASE Statement: Create a program that assigns a salary band (Junior/Mid/Senior/Executive) to employee 7 based on salary ranges of your choice.
3. Simple LOOP: Write a loop that displays "Processing..." 3 times, then "Done!"
4. WHILE LOOP: Calculate how many years it takes for an employee earning \$50,000 to reach \$80,000 with 7% annual raises.
5. FOR LOOP: Display course IDs 101 to 115 with their course names.
6. Cursor FOR Loop: Give a 10% raise to all employees in the Mathematics department. Display the name and new salary of each.
7. Nested Loops: For each course in the Computer Science department, display the course name and list the students enrolled in it with their scores.
8. NULL Handling: Display all enrollments for course 103. Where the score is NULL, display "Not graded".
9. EXIT Statement: Loop through students 1001-1050 and stop at the first one with a GPA below 3.0. Display that student's details.
10. Real Scenario: Create a program that loops through all courses, counts the students enrolled in each, calculates the average score, and displays a warning if the course is more than 80% full (compare with max_students).

Common Mistakes to Avoid

1. Infinite loops: always make sure your loop has a way to exit.
2. Forgetting COMMIT: changes are not saved without COMMIT.
3. NULL comparisons: never use = NULL; always use IS NULL.
4. Missing END LOOP: every LOOP must have an END LOOP.
5. Wrong loop type: use a FOR loop when you know the count and a WHILE loop when you don't.

Tips for Success

- ✓ Start with simple loops before trying nested ones.
- ✓ Use DBMS_OUTPUT to see what is happening inside your loops.
- ✓ Test with small data sets first.
- ✓ Always handle NULL values explicitly.
- ✓ Use meaningful variable names (v_student_count, not v_x).
- ✓ Comment your code to explain the logic.

Chapter 3: Cursors and Advanced Data Processing

In this chapter you will learn:

- What cursors are and why we need them
- The different types of cursors (implicit and explicit)
- How to declare, open, fetch from, and close cursors
- Cursor attributes and how to use them
- Cursor parameters for flexible queries
- Cursor variables (REF CURSORS) for dynamic processing

3.1 Understanding Cursors

What is a Cursor?

A cursor is like a pointer that helps you process database records one by one. Think of it as a bookmark that keeps track of where you are as you read through employee records.

Real-World Analogy

Imagine you have a stack of employee files on your desk. A cursor is like your finger pointing at one file at a time as you process them:

- ✓ You pick up the stack (OPEN the cursor)
- ✓ You read one file at a time (FETCH from the cursor)
- ✓ You process that file's information
- ✓ You move to the next file (FETCH again)
- ✓ When done, you put the stack away (CLOSE the cursor)

Types of Cursors

Cursor Type	What It Is	When Used
Implicit Cursor	Created automatically by Oracle	For single-row SELECT INTO statements and DML statements (INSERT, UPDATE, DELETE)
Explicit Cursor	Declared and controlled by you	When processing multiple rows with full control
Cursor FOR Loop	A special loop for cursors (seen in Chapter 2)	The easiest way to process multiple rows; it handles everything automatically
REF CURSOR	A cursor variable that can be assigned different queries	When you need flexibility in your queries

3.2 Implicit Cursors: Oracle's Automatic Cursors

Every time you use SELECT INTO or a DML statement (INSERT, UPDATE, DELETE), Oracle automatically creates an implicit cursor called SQL.

Implicit Cursor Attributes

Attribute	Returns	Usage
SQL%FOUND	TRUE if the statement affected at least one row	Check whether an UPDATE/DELETE worked
SQL%NOTFOUND	TRUE if the statement affected zero rows	Check whether no rows were found
SQL%ROWCOUNT	Number of rows affected	Count how many rows were processed
SQL%ISOPEN	Always FALSE (implicit cursors close automatically)	Rarely used

Example 1: Using SQL%ROWCOUNT

```
BEGIN
  -- Give a raise to Computer Science staff
  UPDATE employees
  SET salary = salary * 1.10
  WHERE department = 'Computer Science';

  -- Check how many were updated
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Raise given to ' || SQL%ROWCOUNT || ' employees');
  ELSE
    DBMS_OUTPUT.PUT_LINE('No employees found in Computer Science');
  END IF;

  COMMIT;
END;
/
```

Output:

```
Raise given to 16 employees
```

Example 2: Checking Whether a DELETE Worked

```
DECLARE
  v_enrollment_id NUMBER := 999;  -- Enrollment that doesn't exist
BEGIN
  DELETE FROM enrollments
  WHERE enrollment_id = v_enrollment_id;

  IF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('Enrollment ' || v_enrollment_id ||
      ' not found - no deletion');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Enrollment deleted successfully');
    COMMIT;
  END IF;
END;
/
```

3.3 Explicit Cursors: Full Control Over Data Processing

Explicit cursors give you complete control over how you retrieve and process multiple rows.

The Four Steps of Using an Explicit Cursor

Explicit Cursor Lifecycle

```
DECLARE
-- Step 1: DECLARE the cursor
CURSOR cursor_name IS
    SELECT columns FROM table WHERE condition;

-- Declare variables to hold the data
v_variable1 datatype;
v_variable2 datatype;
BEGIN
-- Step 2: OPEN the cursor
OPEN cursor_name;

-- Step 3: FETCH data from the cursor (usually in a loop)
LOOP
    FETCH cursor_name INTO v_variable1, v_variable2;
    EXIT WHEN cursor_name%NOTFOUND;

    -- Process the data
    DBMS_OUTPUT.PUT_LINE(v_variable1 || ' ' || v_variable2);
END LOOP;

-- Step 4: CLOSE the cursor
CLOSE cursor_name;
END;
```

Remember the four steps:

- ✓ DECLARE: define what data you want
- ✓ OPEN: execute the query
- ✓ FETCH: get one row at a time
- ✓ CLOSE: free up resources

Example 3: Basic Explicit Cursor

```
DECLARE
-- Declare the cursor
CURSOR emp_cursor IS
    SELECT employee_id, first_name, last_name, salary
    FROM employees
    WHERE department = 'Business'
    ORDER BY salary DESC;

-- Declare variables to hold the cursor data
v_emp_id      employees.employee_id%TYPE;
v_first_name  employees.first_name%TYPE;
v_last_name   employees.last_name%TYPE;
v_salary      employees.salary%TYPE;
v_count       NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Employees in the Business Department:');
    DBMS_OUTPUT.PUT_LINE('=====');

    -- Open the cursor
    OPEN emp_cursor;

    -- Fetch and process each row
    LOOP
        FETCH emp_cursor INTO v_emp_id, v_first_name, v_last_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        v_count := v_count + 1;
        DBMS_OUTPUT.PUT_LINE(v_count || '. ' || v_first_name || ' ' ||
                               v_last_name || ' (ID: ' || v_emp_id || ') - $' ||
```

```

                                v_salary);

END LOOP;

-- Close the cursor
CLOSE emp_cursor;

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('Total employees: ' || v_count);
END;
/

```

Using %TYPE and %ROWTYPE

Instead of specifying data types yourself, you can use %TYPE to match a column's data type, or %ROWTYPE to match an entire row.

Example 4: Using %ROWTYPE

```

DECLARE
    CURSOR stu_cursor IS
        SELECT student_id, first_name, last_name, gpa, enrollment_date
        FROM students
        WHERE gpa >= 3.8
        ORDER BY gpa DESC, last_name;

    -- %ROWTYPE creates a record with all the cursor's columns
    stu_record stu_cursor%ROWTYPE;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Dean''s List (GPA 3.8 and above):');
    DBMS_OUTPUT.PUT_LINE('=====');

    OPEN stu_cursor;

    LOOP
        FETCH stu_cursor INTO stu_record;
        EXIT WHEN stu_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('Name: ' || stu_record.first_name || ' ' ||
                               stu_record.last_name || ' (ID ' ||
                               stu_record.student_id || ')');
        DBMS_OUTPUT.PUT_LINE('GPA: ' || stu_record.gpa);
        DBMS_OUTPUT.PUT_LINE('Enrolled: ' || stu_record.enrollment_date);
        DBMS_OUTPUT.PUT_LINE('---');
    END LOOP;

    CLOSE stu_cursor;
END;
/

```

3.4 Cursor Attributes

Explicit cursors have attributes that give you information about the cursor's status.

Attribute	Returns	When to Use
cursor_name%FOUND	TRUE if FETCH returned a row	Check whether data was retrieved
cursor_name%NOTFOUND	TRUE if FETCH did not return a row	Exit condition for loops
cursor_name%ROWCOUNT	Number of rows fetched so far	Count rows, limit processing
cursor_name%ISOPEN	TRUE if the cursor is open	Check cursor status

Example 5: Using Cursor Attributes

```
DECLARE
    CURSOR stu_cursor IS
        SELECT first_name, last_name, gpa
        FROM students
        ORDER BY gpa DESC, last_name;

    stu_rec stu_cursor%ROWTYPE;
BEGIN
    -- Check whether the cursor is already open (good practice)
    IF stu_cursor%ISOPEN THEN
        CLOSE stu_cursor;
    END IF;

    OPEN stu_cursor;

    DBMS_OUTPUT.PUT_LINE('Top 5 Students by GPA:');
    DBMS_OUTPUT.PUT_LINE('=====');

    LOOP
        FETCH stu_cursor INTO stu_rec;
        EXIT WHEN stu_cursor%NOTFOUND OR stu_cursor%ROWCOUNT > 5;

        DBMS_OUTPUT.PUT_LINE(stu_cursor%ROWCOUNT || '. ' ||
                               stu_rec.first_name || ' ' || stu_rec.last_name ||
                               ' - ' || stu_rec.gpa);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Rows fetched: ' || stu_cursor%ROWCOUNT);

    CLOSE stu_cursor;
END;
/
```

3.5 Cursor Parameters: Making Cursors Flexible

Cursor parameters allow you to pass values to your cursor, making it reusable with different conditions.

Cursor with Parameters Syntax

```
CURSOR cursor_name (parameter_name datatype) IS
    SELECT columns
    FROM table
    WHERE column = parameter_name;
```

Example 6: Cursor with a Course Parameter

```
DECLARE
    -- Cursor with a parameter
    CURSOR course_cursor (p_course_id NUMBER) IS
        SELECT s.first_name, s.last_name, e.score
        FROM enrollments e
        JOIN students s ON e.student_id = s.student_id
        WHERE e.course_id = p_course_id
        ORDER BY e.score DESC NULLS LAST;

    stu_rec course_cursor%ROWTYPE;

    PROCEDURE display_course_students(p_course_id NUMBER, p_course_name VARCHAR2) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('');
        DBMS_OUTPUT.PUT_LINE('Course: ' || p_course_name);
        DBMS_OUTPUT.PUT_LINE('=====');
```

```

OPEN course_cursor(p_course_id);    -- Pass the parameter

LOOP
    FETCH course_cursor INTO stu_rec;
    EXIT WHEN course_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(stu_rec.first_name || ' ' || stu_rec.last_name ||
                          ' - ' || NVL(TO_CHAR(stu_rec.score), 'Not graded'));

END LOOP;

CLOSE course_cursor;
END;
BEGIN
    -- Use the same cursor for different courses
    display_course_students(101, 'Introduction to Programming');
    display_course_students(102, 'Database Management Systems');
    display_course_students(103, 'PL/SQL Programming');
END;
/

```

Example 7: Cursor with Multiple Parameters

```

DECLARE
    CURSOR gpa_cursor (p_min_gpa NUMBER, p_max_gpa NUMBER) IS
        SELECT student_id, first_name, last_name, gpa, enrollment_date
        FROM students
        WHERE gpa BETWEEN p_min_gpa AND p_max_gpa
        ORDER BY gpa;

    v_count NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Students with a GPA between 3.0 and 3.5:');
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR stu_rec IN gpa_cursor(3.0, 3.5) LOOP
        v_count := v_count + 1;
        DBMS_OUTPUT.PUT_LINE(stu_rec.first_name || ' ' || stu_rec.last_name ||
                              ' (ID: ' || stu_rec.student_id || ') - ' ||
                              stu_rec.gpa);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Total students in this range: ' || v_count);
END;
/

```

3.6 Cursor Variables (REF CURSOR)

A REF CURSOR (cursor variable) is like a pointer to a cursor. Unlike a regular cursor, it can be opened for different SELECT statements at different times.

Why Use a REF CURSOR?

- ✓ When you don't know the query at compile time
- ✓ When you want to return a result set from a procedure or function
- ✓ When you need flexibility in choosing between different queries

REF CURSOR Syntax

```
DECLARE
  -- Declare a cursor type
  TYPE cursor_type_name IS REF CURSOR;

  -- Declare a cursor variable
  cursor_variable cursor_type_name;

  -- Or use the built-in SYS_REFCURSOR
  -- cursor_variable SYS_REFCURSOR;
BEGIN
  -- Open it for different queries
  OPEN cursor_variable FOR SELECT ...;
  -- Process...
  CLOSE cursor_variable;
END;
```

Example 8: Basic REF CURSOR

```
DECLARE
  TYPE emp_cursor_type IS REF CURSOR;
  emp_cv emp_cursor_type;

  v_choice NUMBER := 1;  -- User choice
  emp_rec employees%ROWTYPE;
BEGIN
  -- Different query based on the choice
  IF v_choice = 1 THEN
    DBMS_OUTPUT.PUT_LINE('High Earners (Salary > $80,000):');
    OPEN emp_cv FOR
      SELECT * FROM employees WHERE salary > 80000;
  ELSE
    DBMS_OUTPUT.PUT_LINE('Recent Hires (from 2022):');
    OPEN emp_cv FOR
      SELECT * FROM employees
      WHERE hire_date >= DATE '2022-01-01';
  END IF;

  DBMS_OUTPUT.PUT_LINE('=====');

  LOOP
    FETCH emp_cv INTO emp_rec;
    EXIT WHEN emp_cv%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(emp_rec.first_name || ' ' || emp_rec.last_name ||
      ' - $' || emp_rec.salary ||
      ' (Hired: ' || emp_rec.hire_date || ')');
  END LOOP;

  CLOSE emp_cv;
END;
/
```

Example 9: Dynamic Query with a REF CURSOR

```
DECLARE
  emp_cv SYS_REFCURSOR;
  v_department VARCHAR2(50) := 'Computer Science';
  v_min_salary NUMBER := 70000;
  v_sql VARCHAR2(500);

  v_emp_id NUMBER;
  v_name VARCHAR2(100);
  v_salary NUMBER;
BEGIN
  -- Build a dynamic query
  v_sql := 'SELECT employee_id, first_name || ' ' || last_name, salary ' ||
```

```

        'FROM employees ' ||
        'WHERE department = :dept AND salary > :min_sal ' ||
        'ORDER BY salary DESC';

DBMS_OUTPUT.PUT_LINE('Employees in ' || v_department ||
                      ' earning over $' || v_min_salary || ':');
DBMS_OUTPUT.PUT_LINE('=====');

-- Open the cursor with bind variables
OPEN emp_cv FOR v_sql USING v_department, v_min_salary;

LOOP
    FETCH emp_cv INTO v_emp_id, v_name, v_salary;
    EXIT WHEN emp_cv%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('ID: ' || v_emp_id || ' - ' || v_name ||
                          ' ($' || v_salary || ')');
END LOOP;

CLOSE emp_cv;
END;
/

```

3.7 Advanced Cursor Techniques

Cursor FOR UPDATE: Locking Rows for Update

Use FOR UPDATE when you want to lock rows so that nobody else can modify them while you are processing them.

Example 10: Cursor FOR UPDATE

```

DECLARE
    CURSOR emp_cursor IS
        SELECT employee_id, first_name, last_name, salary
        FROM employees
        WHERE department = 'Engineering'
        FOR UPDATE OF salary NOWAIT;    -- Lock the rows

    v_new_salary NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Updating Engineering Department Salaries:');
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR emp_rec IN emp_cursor LOOP
        -- Calculate a 10% raise
        v_new_salary := emp_rec.salary * 1.10;

        -- Update using CURRENT OF
        UPDATE employees
        SET salary = v_new_salary
        WHERE CURRENT OF emp_cursor;

        DBMS_OUTPUT.PUT_LINE(emp_rec.first_name || ' ' || emp_rec.last_name ||
                              ': $' || emp_rec.salary || ' -> $' ||
                              ROUND(v_new_salary, 2));
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('All salaries updated successfully');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: Rows might be locked by another user');
        ROLLBACK;

```

```
END;
/
```

Cursor with a JOIN

Example 11: Cursor over Multiple Tables

```
DECLARE
    CURSOR top_scores_cursor IS
        SELECT s.first_name || ' ' || s.last_name AS student_name,
               c.course_code,
               c.course_name,
               c.instructor,
               e.score,
               e.grade
        FROM enrollments e
        JOIN students s ON e.student_id = s.student_id
        JOIN courses c ON e.course_id = c.course_id
        WHERE e.score >= 90
        ORDER BY c.course_code, e.score DESC;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Top Scores (90 and above) by Course:');
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR rec IN top_scores_cursor LOOP
        DBMS_OUTPUT.PUT_LINE('Course: ' || rec.course_code || ' - ' || rec.course_name);
        DBMS_OUTPUT.PUT_LINE('  Student: ' || rec.student_name);
        DBMS_OUTPUT.PUT_LINE('  Instructor: ' || rec.instructor);
        DBMS_OUTPUT.PUT_LINE('  Score: ' || rec.score || ' (' || rec.grade || ')');
        DBMS_OUTPUT.PUT_LINE('  ---');
    END LOOP;
END;
/
```

3.8 Best Practices and Common Pitfalls

Common Mistakes to Avoid

1. Forgetting to CLOSE a cursor: always close explicit cursors to free memory.
2. Fetching after the cursor is closed: check %ISOPEN before fetching.
3. Not checking %NOTFOUND: always have an exit condition.
4. Opening a cursor inside a loop: open once, fetch many times.
5. Using SELECT INTO for multiple rows: use a cursor instead.

Best Practices

- ✓ Use a cursor FOR loop when possible; it is simpler.
- ✓ Use %ROWTYPE to avoid declaring individual variables.
- ✓ Use parameters to make cursors reusable.
- ✓ Always include exception handling.
- ✓ Use meaningful cursor names (emp_salary_cursor, not c1).
- ✓ Close cursors in the EXCEPTION block too.

3.9 Practical Application: Complete Department Report

Example 12: Comprehensive Department Report

```
DECLARE
    -- Cursor for departments
```

```

CURSOR dept_cursor IS
    SELECT department_id, department_name, budget
    FROM departments
    ORDER BY department_name;

-- Cursor for employees (with a parameter)
CURSOR emp_cursor (p_department VARCHAR2) IS
    SELECT employee_id, first_name, last_name, salary, hire_date,
           ROUND(MONTHS_BETWEEN(SYSDATE, hire_date) / 12, 1) AS years_service
    FROM employees
    WHERE department = p_department
    ORDER BY salary DESC;

v_emp_count    NUMBER;
v_total_salary NUMBER;
v_avg_salary    NUMBER;
v_max_salary    NUMBER;
v_min_salary    NUMBER;
v_avg_service   NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('          COMPREHENSIVE DEPARTMENT REPORT');
    DBMS_OUTPUT.PUT_LINE('=====');

    FOR dept_rec IN dept_cursor LOOP
        -- Calculate department statistics
        SELECT COUNT(*),
               NVL(SUM(salary), 0),
               NVL(AVG(salary), 0),
               NVL(MAX(salary), 0),
               NVL(MIN(salary), 0),
               NVL(AVG(MONTHS_BETWEEN(SYSDATE, hire_date) / 12), 0)
        INTO v_emp_count, v_total_salary, v_avg_salary,
             v_max_salary, v_min_salary, v_avg_service
        FROM employees
        WHERE department = dept_rec.department_name;

        -- Skip departments with no employees
        IF v_emp_count = 0 THEN
            CONTINUE;
        END IF;

        -- Department header
        DBMS_OUTPUT.PUT_LINE('');
        DBMS_OUTPUT.PUT_LINE('Department: ' || dept_rec.department_name);
        DBMS_OUTPUT.PUT_LINE('-----');
        DBMS_OUTPUT.PUT_LINE('Statistics:');
        DBMS_OUTPUT.PUT_LINE('  Total Employees: ' || v_emp_count);
        DBMS_OUTPUT.PUT_LINE('  Total Payroll: $' || ROUND(v_total_salary, 2));
        DBMS_OUTPUT.PUT_LINE('  Average Salary: $' || ROUND(v_avg_salary, 2));
        DBMS_OUTPUT.PUT_LINE('  Highest Salary: $' || ROUND(v_max_salary, 2));
        DBMS_OUTPUT.PUT_LINE('  Lowest Salary: $' || ROUND(v_min_salary, 2));
        DBMS_OUTPUT.PUT_LINE('  Avg Years Service: ' || ROUND(v_avg_service, 1));
        DBMS_OUTPUT.PUT_LINE('  Department Budget: $' || dept_rec.budget);
        DBMS_OUTPUT.PUT_LINE('');
        DBMS_OUTPUT.PUT_LINE('Employee Details:');

        -- List all employees
        FOR emp_rec IN emp_cursor(dept_rec.department_name) LOOP
            DBMS_OUTPUT.PUT_LINE('  ' || emp_rec.first_name || ' ' ||
                                emp_rec.last_name);
            DBMS_OUTPUT.PUT_LINE('    ID: ' || emp_rec.employee_id ||
                                ' | Salary: $' || emp_rec.salary ||
                                ' | Service: ' || emp_rec.years_service ||
                                ' years');
        END LOOP;

        -- Add a warning if the payroll is above the department budget
        IF v_total_salary > dept_rec.budget THEN
            DBMS_OUTPUT.PUT_LINE('  WARNING: Payroll exceeds budget by $' ||

```

```

                                (v_total_salary - dept_rec.budget));
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('                                REPORT COMPLETED');
    DBMS_OUTPUT.PUT_LINE('=====');
END;
/

```

Summary of Chapter 3

1. Cursor fundamentals: what cursors are and why we need them.
2. Implicit cursors: automatic cursors for DML and single-row SELECT (SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT).
3. Explicit cursors: full control with DECLARE, OPEN, FETCH, CLOSE.
4. Cursor attributes: %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN.
5. %TYPE and %ROWTYPE: easy variable declarations that match the table structure.
6. Cursor parameters: making cursors flexible and reusable.
7. REF CURSOR: cursor variables for dynamic queries.
8. FOR UPDATE: locking rows for safe updates.

When to Use Each Cursor Type

Situation	Best Choice	Why
Processing multiple rows simply	Cursor FOR loop	Automatic handling, cleanest code
Need fine control over fetching	Explicit cursor	You control when to fetch and how many rows
Query not known until runtime	REF CURSOR	Dynamic SQL capability
Single row expected	SELECT INTO	Simplest approach
Need to update the fetched rows	Cursor FOR UPDATE	Locks rows for safe updates

Complete Syntax Reference

```

-- Implicit cursor attributes
IF SQL%FOUND THEN ...
IF SQL%NOTFOUND THEN ...
v_count := SQL%ROWCOUNT;

-- Explicit cursor (traditional)
DECLARE
    CURSOR cursor_name IS SELECT ...;
    v_variable datatype;
BEGIN
    OPEN cursor_name;
    LOOP
        FETCH cursor_name INTO v_variable;
        EXIT WHEN cursor_name%NOTFOUND;
        -- process
    END LOOP;
    CLOSE cursor_name;
END;

-- Cursor FOR loop (easiest)

```

```

FOR record IN cursor_name LOOP
    -- use record.column_name
END LOOP;

-- Cursor with parameters
CURSOR cursor_name (param1 datatype, param2 datatype) IS
    SELECT ... WHERE column = param1 ...;

-- Using the parameterized cursor
OPEN cursor_name(value1, value2);
-- or
FOR rec IN cursor_name(value1, value2) LOOP ...

-- REF CURSOR
DECLARE
    TYPE ref_cursor_type IS REF CURSOR;
    cv ref_cursor_type; -- or use SYS_REFCURSOR
BEGIN
    OPEN cv FOR SELECT ...;
    LOOP
        FETCH cv INTO ...;
        EXIT WHEN cv%NOTFOUND;
    END LOOP;
    CLOSE cv;
END;

-- Cursor FOR UPDATE
CURSOR cursor_name IS
    SELECT ... FOR UPDATE OF column NOWAIT;

UPDATE table SET ... WHERE CURRENT OF cursor_name;

```

Practice Exercises

1. Basic Explicit Cursor: Create a cursor to display all courses in the Computer Science department with their codes and credits. Use the traditional OPEN-FETCH-CLOSE method.
2. Cursor Attributes: Write a program that sets the status of all enrollments in course 110 to 'Completed'. Use SQL%ROWCOUNT to display how many enrollments were updated.
3. %ROWTYPE: Create a cursor for students with a GPA below 3.0. Use %ROWTYPE to fetch entire rows and display all the information.
4. Cursor with Parameter: Create a cursor that accepts a student ID as a parameter and lists that student's courses and scores. Use it for students 1001, 1002, and 1003.
5. Multiple Parameters: Create a cursor with minimum and maximum score parameters. Display all enrollments (student name, course name, score) within that range.
6. REF CURSOR: Write a program that uses a REF CURSOR. Based on a choice variable, show either students who enrolled on 2023-09-01 or students with a GPA of 3.8 and above.
7. Nested Cursors: Create a report showing each course with its enrolled students. Use one cursor for courses and another (with a parameter) for the students.
8. Cursor FOR UPDATE: Give a 12% raise to all employees in the Psychology department. Use FOR UPDATE and CURRENT OF. Display the salaries before and after.
9. Cursor with JOIN: Create a cursor that joins the EMPLOYEES and DEPARTMENTS tables (on department name). Display the employee name, job title, department location, and salary for employees earning over \$75,000.
10. Complete Application: Create a course performance program that uses a cursor with a course parameter, calculates the minimum, maximum, and average score for each course, lists all students in the course, and identifies students scoring below the course average.

Common Errors and Solutions

Error	Cause	Solution
ORA-01001: invalid cursor	Trying to fetch from a closed cursor	Make sure the cursor is open before fetching
ORA-06511: cursor already open	Trying to open a cursor that is already open	Check %ISOPEN before opening
ORA-01422: exact fetch returns more than requested number of rows	Using SELECT INTO for multiple rows	Use a cursor instead of SELECT INTO
Infinite loop in cursor	Forgetting EXIT WHEN %NOTFOUND	Always include an exit condition
Memory leak	Not closing cursors	Always CLOSE the cursor, even in the exception handler

Performance Tips

- ✓ Use a cursor FOR loop when you don't need fine control; it is optimized.
- ✓ Fetch in batches using BULK COLLECT for large data sets (covered in advanced topics).
- ✓ Close cursors as soon as you are done to free memory.
- ✓ Use cursor parameters instead of building dynamic SQL when possible.
- ✓ Avoid opening cursors inside loops; open once and use parameters.
- ✓ Use indexes on the columns in the WHERE clause of the cursor query.

Quick Decision Guide: Which Cursor Approach Should I Use?

Use a **cursor FOR loop** when:

- ✓ You need to process all rows
- ✓ The processing is simple and straightforward
- ✓ You don't need the row count during processing

Use an **explicit cursor** when:

- ✓ You need to control exactly when to fetch
- ✓ You want to stop fetching based on conditions
- ✓ You need cursor attributes such as %ROWCOUNT during processing

Use a **REF CURSOR** when:

- ✓ The query is built dynamically
- ✓ You are returning a result set from a procedure or function
- ✓ Different queries are needed based on runtime conditions

Use **SELECT INTO** when:

- ✓ You expect exactly ONE row
- ✓ You will handle the NO_DATA_FOUND exception

Chapter 4: Procedures and Functions: Building Reusable Code

In this chapter you will learn:

- ✓ What procedures and functions are
- ✓ How to create and use stored procedures
- ✓ How to create and use functions
- ✓ Parameter modes: IN, OUT, IN OUT
- ✓ The differences between procedures and functions
- ✓ How to organize and reuse code effectively

4.1 Introduction to Subprograms

Procedures and functions are named PL/SQL blocks that are stored in the database. They allow you to write code once and use it many times.

Real-World Analogy

Think of procedures and functions like recipes in a cookbook:

- ✓ Without them: every time you want to make a cake, you write down all the steps from scratch.
- ✓ With them: you write the recipe once, give it a name ("Chocolate Cake"), and anyone can use it simply by calling its name.

Why Use Procedures and Functions?

Problem: repetitive code. Imagine you need to calculate employee bonuses in three different places in your application:

```
-- In program 1
v_bonus := salary * 0.10;

-- In program 2
v_bonus := salary * 0.10;

-- In program 3
v_bonus := salary * 0.10;
```

If the bonus rule changes to 12%, you have to change it in three places.

Solution: create a function.

```
-- Write once
CREATE FUNCTION calculate_bonus(p_salary NUMBER)
RETURN NUMBER IS
BEGIN
    RETURN p_salary * 0.12;
END;

-- Use everywhere
v_bonus := calculate_bonus(salary);
```

Change the rule in one place and it is updated everywhere.

Procedures vs Functions

Aspect	Procedure	Function
Purpose	Performs an action	Calculates and returns a value
Return value	Does not return a value (uses OUT parameters instead)	Must return exactly one value
Usage	Called as a standalone statement	Called as part of an expression
Example	Give an employee a raise, send an email, update records	Calculate tax, get a full name, check eligibility
Can be used in SQL?	No	Yes (with restrictions)

4.2 Creating and Using Procedures

Creating a Procedure

```
CREATE OR REPLACE PROCEDURE procedure_name
(
    parameter1 IN      datatype,
    parameter2 OUT     datatype,
    parameter3 IN OUT  datatype
)
IS
    -- Variable declarations (optional)
    variable_name datatype;
BEGIN
    -- Executable statements
    statements;
EXCEPTION
    -- Exception handling (optional)
    WHEN exception_name THEN
        statements;
END procedure_name;
/
```

Calling a Procedure

```
-- From a PL/SQL block
BEGIN
    procedure_name(value1, value2, value3);
END;
/

-- Or directly
EXECUTE procedure_name(value1, value2, value3);
```

Example 1: Simple Procedure with No Parameters

```
CREATE OR REPLACE PROCEDURE display_university_info
IS
    v_student_count NUMBER;
    v_course_count   NUMBER;
    v_emp_count      NUMBER;
    v_dept_count     NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_student_count FROM students;
    SELECT COUNT(*) INTO v_course_count FROM courses;
    SELECT COUNT(*) INTO v_emp_count FROM employees;
    SELECT COUNT(*) INTO v_dept_count FROM departments;

    DBMS_OUTPUT.PUT_LINE('===== UNIVERSITY INFORMATION =====');
    DBMS_OUTPUT.PUT_LINE('Total Students: ' || v_student_count);
```

```

        DBMS_OUTPUT.PUT_LINE('Total Courses: ' || v_course_count);
        DBMS_OUTPUT.PUT_LINE('Total Staff: ' || v_emp_count);
        DBMS_OUTPUT.PUT_LINE('Total Departments: ' || v_dept_count);
        DBMS_OUTPUT.PUT_LINE('=====');
END display_university_info;
/

-- How to call it
BEGIN
    display_university_info;
END;
/

-- Or simply
EXEC display_university_info;

```

Example 2: Procedure with IN Parameters

```

CREATE OR REPLACE PROCEDURE give_raise
(
    p_emp_id          IN NUMBER,
    p_raise_percent IN NUMBER
)
IS
    v_emp_name  VARCHAR2(100);
    v_old_salary NUMBER;
    v_new_salary NUMBER;
BEGIN
    -- Get current salary
    SELECT first_name || ' ' || last_name, salary
    INTO v_emp_name, v_old_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Calculate new salary
    v_new_salary := v_old_salary * (1 + p_raise_percent / 100);

    -- Update salary
    UPDATE employees
    SET salary = v_new_salary
    WHERE employee_id = p_emp_id;

    COMMIT;

    -- Display result
    DBMS_OUTPUT.PUT_LINE('Raise given to: ' || v_emp_name);
    DBMS_OUTPUT.PUT_LINE('Old Salary: $' || v_old_salary);
    DBMS_OUTPUT.PUT_LINE('New Salary: $' || ROUND(v_new_salary, 2));
    DBMS_OUTPUT.PUT_LINE('Increase: ' || p_raise_percent || '%');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID ' || p_emp_id || ' not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        ROLLBACK;
END give_raise;
/

-- How to call it
BEGIN
    give_raise(5, 10);    -- Give employee 5 a 10% raise
END;
/

```

4.3 Parameter Modes: IN, OUT, and IN OUT

Mode	Direction	Can Modify?	Default Value?	Usage
IN	Into the procedure	No (read-only)	Yes	Pass data to the procedure
OUT	Out of the procedure	Yes (write)	No	Return data from the procedure
IN OUT	Both ways	Yes (read and write)	No	Pass data in and get the modified data back

Example 3: Procedure with OUT Parameters

```
CREATE OR REPLACE PROCEDURE get_employee_info
(
    p_emp_id    IN  NUMBER,
    p_name      OUT VARCHAR2,
    p_salary    OUT  NUMBER,
    p_department OUT VARCHAR2,
    p_location  OUT  VARCHAR2
)
IS
BEGIN
    SELECT e.first_name || ' ' || e.last_name,
           e.salary,
           e.department,
           d.location
    INTO p_name, p_salary, p_department, p_location
    FROM employees e
    JOIN departments d ON e.department = d.department_name
    WHERE e.employee_id = p_emp_id;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_name      := 'NOT FOUND';
        p_salary    := 0;
        p_department := 'N/A';
        p_location  := 'N/A';
END get_employee_info;
/

-- How to use OUT parameters
DECLARE
    v_name      VARCHAR2(100);
    v_salary    NUMBER;
    v_dept      VARCHAR2(50);
    v_location  VARCHAR2(50);
BEGIN
    get_employee_info(1, v_name, v_salary, v_dept, v_location);

    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary: $' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept);
    DBMS_OUTPUT.PUT_LINE('Office: ' || v_location);
END;
/
```

Example 4: Procedure with an IN OUT Parameter

```
CREATE OR REPLACE PROCEDURE apply_tax
(
    p_amount    IN OUT NUMBER,      -- Will be modified
    p_tax_rate  IN    NUMBER DEFAULT 10
)
IS
    v_tax NUMBER;
```

```

BEGIN
    v_tax := p_amount * (p_tax_rate / 100);
    p_amount := p_amount - v_tax;      -- Modify the parameter

    DBMS_OUTPUT.PUT_LINE('Tax deducted: $' || ROUND(v_tax, 2));
    DBMS_OUTPUT.PUT_LINE('Amount after tax: $' || ROUND(p_amount, 2));
END apply_tax;
/

-- Using the IN OUT parameter
DECLARE
    v_salary NUMBER := 60000;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Original amount: $' || v_salary);

    apply_tax(v_salary, 15);    -- 15% tax

    DBMS_OUTPUT.PUT_LINE('Final amount: $' || v_salary);
END;
/

```

Output:

```

Original amount: $60000
Tax deducted: $9000
Amount after tax: $51000
Final amount: $51000

```

4.4 Creating and Using Functions

Creating a Function

```

CREATE OR REPLACE FUNCTION function_name
(
    parameter1 IN datatype,
    parameter2 IN datatype
)
RETURN return_datatype
IS
    -- Variable declarations
    variable_name datatype;
BEGIN
    -- Executable statements
    statements;

    RETURN return_value;    -- A function must RETURN a value
EXCEPTION
    WHEN exception_name THEN
        RETURN error_value;
END function_name;
/

```

Using a Function

```

-- In an assignment
v_result := function_name(value1, value2);

-- In a SQL statement
SELECT function_name(column) FROM table;

-- In a condition
IF function_name(value) > 100 THEN ...

```

Example 5: Simple Function

Salaries in the EMPLOYEES table are annual amounts. This function converts them to a monthly figure.

```
CREATE OR REPLACE FUNCTION calculate_monthly_salary
(
    p_annual_salary IN NUMBER
)
RETURN NUMBER
IS
BEGIN
    RETURN ROUND(p_annual_salary / 12, 2);
END calculate_monthly_salary;
/

-- Using the function
DECLARE
    v_annual NUMBER := 60000;
    v_monthly NUMBER;
BEGIN
    v_monthly := calculate_monthly_salary(v_annual);

    DBMS_OUTPUT.PUT_LINE('Annual: $' || v_annual);
    DBMS_OUTPUT.PUT_LINE('Monthly: $' || v_monthly);
END;
/

-- Using it in a SQL query
SELECT first_name,
       salary AS annual,
       calculate_monthly_salary(salary) AS monthly
FROM employees
WHERE employee_id = 1;
```

Example 6: Function Returning Employee Status

```
CREATE OR REPLACE FUNCTION get_employee_status
(
    p_emp_id IN NUMBER
)
RETURN VARCHAR2
IS
    v_years_service NUMBER;
    v_salary NUMBER;
    v_status VARCHAR2(20);
BEGIN
    -- Get employee information
    SELECT ROUND(MONTHS_BETWEEN(SYSDATE, hire_date) / 12, 1),
           salary
    INTO v_years_service, v_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Determine status
    IF v_years_service > 8 AND v_salary > 90000 THEN
        v_status := 'SENIOR';
    ELSIF v_years_service > 5 OR v_salary > 75000 THEN
        v_status := 'INTERMEDIATE';
    ELSE
        v_status := 'JUNIOR';
    END IF;

    RETURN v_status;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'NOT FOUND';
    WHEN OTHERS THEN
        RETURN 'ERROR';
END get_employee_status;
```

```

/

-- Using the function
BEGIN
  FOR emp IN (SELECT employee_id, first_name, last_name
              FROM employees
              WHERE employee_id BETWEEN 1 AND 7) LOOP

    DBMS_OUTPUT.PUT_LINE(emp.first_name || ' ' || emp.last_name ||
                          ' - Status: ' || get_employee_status(emp.employee_id));

  END LOOP;
END;
/

```

4.5 Default Parameter Values

You can specify default values for IN parameters. If the caller doesn't provide a value, the default is used.

Example 7: Function with Default Parameters

```

CREATE OR REPLACE FUNCTION calculate_bonus
(
  p_salary          IN NUMBER,
  p_performance_rating IN NUMBER DEFAULT 3,  -- Default value
  p_years_service   IN NUMBER DEFAULT 0
)
RETURN NUMBER
IS
  v_bonus NUMBER;
BEGIN
  -- Base bonus on performance
  v_bonus := p_salary * (p_performance_rating * 0.05);

  -- Add a loyalty bonus
  IF p_years_service > 5 THEN
    v_bonus := v_bonus + (p_salary * 0.10);
  END IF;

  RETURN v_bonus;
END calculate_bonus;
/

-- Different ways to call it
DECLARE
  v_salary NUMBER := 60000;
BEGIN
  -- Using all parameters
  DBMS_OUTPUT.PUT_LINE('Full call: $' || calculate_bonus(v_salary, 4, 6));

  -- Using defaults (performance = 3, years = 0)
  DBMS_OUTPUT.PUT_LINE('Default params: $' || calculate_bonus(v_salary));

  -- Using named notation
  DBMS_OUTPUT.PUT_LINE('Named params: $' ||
    calculate_bonus(p_salary => v_salary, p_years_service => 7));
END;
/

```

4.6 Positional vs Named Notation

You can pass parameters to procedures and functions in two ways, or mix them.

Method	Syntax	When to Use
Positional	procedure_name(value1, value2, value3)	When you pass all parameters in order
Named	procedure_name(param1 => value1, param2 => value2)	When you want to skip parameters or pass them in a different order
Mixed	procedure_name(value1, param3 => value3)	Positional first, then named

Example 8: Benefits of Named Notation

```

CREATE OR REPLACE PROCEDURE create_employee_report
(
    p_department      IN VARCHAR2 DEFAULT NULL,
    p_min_salary      IN NUMBER   DEFAULT 0,
    p_max_salary      IN NUMBER   DEFAULT 999999,
    p_include_managers IN BOOLEAN  DEFAULT TRUE
)
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Generating report...');
    DBMS_OUTPUT.PUT_LINE('Department: ' || NVL(p_department, 'ALL'));
    DBMS_OUTPUT.PUT_LINE('Salary Range: $' || p_min_salary ||
        ' - $' || p_max_salary);
END create_employee_report;
/

-- Different ways to call it
BEGIN
    -- Positional (must pass all in order)
    create_employee_report('Business', 50000, 70000, FALSE);

    -- Named (can skip parameters)
    create_employee_report(p_department => 'Engineering', p_min_salary => 60000);

    -- Mixed (positional, then named)
    create_employee_report('Business', p_max_salary => 60000);

    -- Only one specific parameter
    create_employee_report(p_department => 'Mathematics');
END;
/

```

4.7 Viewing and Managing Stored Procedures and Functions

Useful Data Dictionary Views

```
-- View all your procedures and functions
SELECT object_name, object_type, status
FROM user_objects
WHERE object_type IN ('PROCEDURE', 'FUNCTION')
ORDER BY object_name;

-- View the source code
SELECT text
FROM user_source
WHERE name = 'GIVE_RAISE'
ORDER BY line;

-- Check for compilation errors
SELECT line, position, text
FROM user_errors
WHERE name = 'GIVE_RAISE'
ORDER BY line, position;
```

Dropping Procedures and Functions

```
-- Drop a procedure
DROP PROCEDURE give_raise;

-- Drop a function
DROP FUNCTION calculate_bonus;

-- Check whether it exists before dropping
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM user_objects
    WHERE object_name = 'GIVE_RAISE'
    AND object_type = 'PROCEDURE';

    IF v_count > 0 THEN
        EXECUTE IMMEDIATE 'DROP PROCEDURE give_raise';
        DBMS_OUTPUT.PUT_LINE('Procedure dropped');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Procedure does not exist');
    END IF;
END;
/
```

4.8 Real-World Examples

Example 9: Staff Management Routines

```
-- Function: calculate total compensation
CREATE OR REPLACE FUNCTION get_total_compensation
(
    p_emp_id IN NUMBER
)
RETURN NUMBER
IS
    v_salary      NUMBER;
    v_commission  NUMBER;
BEGIN
    SELECT salary, NVL(commission_pct, 0) * salary
    INTO v_salary, v_commission
    FROM employees
    WHERE employee_id = p_emp_id;
```

```

        RETURN v_salary + v_commission;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END get_total_compensation;
/

-- Procedure: promote an employee
CREATE OR REPLACE PROCEDURE promote_employee
(
    p_emp_id          IN NUMBER,
    p_new_job_title    IN VARCHAR2,
    p_salary_increase_pct IN NUMBER DEFAULT 15
)
IS
    v_emp_name  VARCHAR2(100);
    v_old_salary NUMBER;
    v_new_salary NUMBER;
    v_old_job    VARCHAR2(50);
BEGIN
    -- Get current information
    SELECT first_name || ' ' || last_name, salary, job_title
    INTO v_emp_name, v_old_salary, v_old_job
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Calculate new salary
    v_new_salary := v_old_salary * (1 + p_salary_increase_pct / 100);

    -- Update the employee
    UPDATE employees
    SET job_title = p_new_job_title,
        salary     = v_new_salary
    WHERE employee_id = p_emp_id;

    COMMIT;

    -- Report the promotion
    DBMS_OUTPUT.PUT_LINE('==== EMPLOYEE PROMOTION ====');
    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp_name);
    DBMS_OUTPUT.PUT_LINE('Old Job: ' || v_old_job || ' -> New Job: ' || p_new_job_title);
    DBMS_OUTPUT.PUT_LINE('Old Salary: $' || v_old_salary ||
        ' -> New Salary: $' || ROUND(v_new_salary, 2));
    DBMS_OUTPUT.PUT_LINE('Increase: ' || p_salary_increase_pct || '%');
    DBMS_OUTPUT.PUT_LINE('=====');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        ROLLBACK;
END promote_employee;
/

-- Function: check whether an employee is eligible for a raise
CREATE OR REPLACE FUNCTION is_eligible_for_raise
(
    p_emp_id    IN NUMBER,
    p_min_years IN NUMBER DEFAULT 1
)
RETURN BOOLEAN
IS
    v_years_service  NUMBER;
    v_last_raise_date DATE;
BEGIN
    -- The EMPLOYEES table has no last_raise_date column,
    -- so hire_date is used to simulate it here
    SELECT MONTHS_BETWEEN(SYSDATE, hire_date) / 12,
        hire_date

```

```

        INTO v_years_service, v_last_raise_date
        FROM employees
        WHERE employee_id = p_emp_id;

        -- Eligible if worked the minimum years and no raise in the last 12 months
        IF v_years_service >= p_min_years AND
            MONTHS_BETWEEN(SYSDATE, v_last_raise_date) >= 12 THEN
            RETURN TRUE;
        ELSE
            RETURN FALSE;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN FALSE;
    END is_eligible_for_raise;
/

-- Using them together
BEGIN
    -- Check eligibility and give raises
    FOR emp IN (SELECT employee_id, first_name, last_name
                FROM employees
                WHERE department = 'Computer Science') LOOP

        IF is_eligible_for_raise(emp.employee_id) THEN
            give_raise(emp.employee_id, 10);
        ELSE
            DBMS_OUTPUT.PUT_LINE(emp.first_name || ' ' || emp.last_name ||
                                  ' - Not eligible for raise yet');
        END IF;
    END LOOP;
END;
/

```

Example 10: Department Statistics

```

-- Function: get the department average salary
CREATE OR REPLACE FUNCTION get_dept_avg_salary
(
    p_department IN VARCHAR2
)
RETURN NUMBER
IS
    v_avg_salary NUMBER;
BEGIN
    SELECT AVG(salary)
    INTO v_avg_salary
    FROM employees
    WHERE department = p_department;

    RETURN NVL(v_avg_salary, 0);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END get_dept_avg_salary;
/

-- Procedure: generate a department report
CREATE OR REPLACE PROCEDURE generate_dept_report
(
    p_dept_id IN NUMBER
)
IS
    v_dept_name      VARCHAR2(50);
    v_location       VARCHAR2(50);
    v_budget         NUMBER;
    v_manager_name   VARCHAR2(100);
    v_emp_count      NUMBER;
    v_course_count   NUMBER;
    v_total_salary   NUMBER;

```

```

        v_avg_salary      NUMBER;
BEGIN
    -- Get department info (the manager may be NULL, hence the LEFT JOIN)
    SELECT d.department_name, d.location, d.budget,
           e.first_name || ' ' || e.last_name
    INTO v_dept_name, v_location, v_budget, v_manager_name
    FROM departments d
    LEFT JOIN employees e ON d.manager_id = e.employee_id
    WHERE d.department_id = p_dept_id;

    -- Staff statistics
    SELECT COUNT(*), NVL(SUM(salary), 0)
    INTO v_emp_count, v_total_salary
    FROM employees
    WHERE department = v_dept_name;

    -- Number of courses offered by the department
    SELECT COUNT(*) INTO v_course_count
    FROM courses
    WHERE department = v_dept_name;

    -- Reuse the function
    v_avg_salary := get_dept_avg_salary(v_dept_name);

    DBMS_OUTPUT.PUT_LINE('==== DEPARTMENT REPORT ====');
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept_name);
    DBMS_OUTPUT.PUT_LINE('Location: ' || v_location);
    DBMS_OUTPUT.PUT_LINE('Manager: ' || NVL(TRIM(v_manager_name), 'None'));
    DBMS_OUTPUT.PUT_LINE('Courses Offered: ' || v_course_count);
    DBMS_OUTPUT.PUT_LINE('Staff: ' || v_emp_count);
    DBMS_OUTPUT.PUT_LINE('Total Payroll: $' || v_total_salary);
    DBMS_OUTPUT.PUT_LINE('Average Salary: $' || ROUND(v_avg_salary, 2));
    DBMS_OUTPUT.PUT_LINE('Budget: $' || v_budget);
    DBMS_OUTPUT.PUT_LINE('=====');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Department ' || p_dept_id || ' not found');
END generate_dept_report;
/

-- Using the procedure
BEGIN
    generate_dept_report(10);    -- Computer Science
END;
/

```

Example 11: Validation Functions

```

-- Function to validate an email address
CREATE OR REPLACE FUNCTION is_valid_email
(
    p_email IN VARCHAR2
)
RETURN BOOLEAN
IS
BEGIN
    -- Simple validation: must contain @ and .
    IF p_email LIKE '%_@_%._%' THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END is_valid_email;
/

-- Function to check whether a course still has free seats
CREATE OR REPLACE FUNCTION has_available_seat
(
    p_course_id IN NUMBER
)

```

```

RETURN BOOLEAN
IS
    v_max_students NUMBER;
    v_enrolled      NUMBER;
BEGIN
    SELECT max_students INTO v_max_students
    FROM courses
    WHERE course_id = p_course_id;

    SELECT COUNT(*) INTO v_enrolled
    FROM enrollments
    WHERE course_id = p_course_id
    AND status = 'Active';

    RETURN v_enrolled < v_max_students;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE; -- Course does not exist
END has_available_seat;
/

-- Using the validation functions
DECLARE
    v_email      VARCHAR2(100) := 'new.student@email.com';
    v_course_id  NUMBER := 110; -- Machine Learning
BEGIN
    IF is_valid_email(v_email) THEN
        DBMS_OUTPUT.PUT_LINE('Email is valid');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Email is invalid');
    END IF;

    IF has_available_seat(v_course_id) THEN
        DBMS_OUTPUT.PUT_LINE('Course ' || v_course_id || ' has free seats');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Course ' || v_course_id || ' is full or does not exist');
    END IF;
END;
/

```

Example 12: Audit Trail Pattern

```

-- Create the audit table first
-- CREATE TABLE salary_audit (
--     audit_id      NUMBER PRIMARY KEY,
--     employee_id   NUMBER,
--     old_salary     NUMBER,
--     new_salary     NUMBER,
--     changed_by    VARCHAR2(50),
--     changed_date   DATE
-- );

-- Procedure with an audit trail
CREATE OR REPLACE PROCEDURE update_salary_with_audit
(
    p_emp_id      IN NUMBER,
    p_new_salary  IN NUMBER,
    p_changed_by  IN VARCHAR2
)
IS
    v_old_salary  NUMBER;
BEGIN
    -- Get current salary
    SELECT salary INTO v_old_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Update salary
    UPDATE employees
    SET salary = p_new_salary

```

```

WHERE employee_id = p_emp_id;

-- Create the audit record (simulated here with output)
DBMS_OUTPUT.PUT_LINE('AUDIT: Employee ' || p_emp_id ||
    ' salary changed from $' || v_old_salary ||
    ' to $' || p_new_salary ||
    ' by ' || p_changed_by);

COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END update_salary_with_audit;
/

```

Summary of Chapter 4

1. Subprogram basics: what procedures and functions are, why we use them (reusability, maintainability), and how they differ.
2. Procedures: creating stored procedures, calling them with EXECUTE or from PL/SQL blocks, and the parameter modes IN, OUT, and IN OUT.
3. Functions: creating and using functions, the RETURN statement requirement, and using functions in SQL queries.
4. Parameters: default values for flexibility, positional vs named notation, and when to use each parameter mode.
5. Good practice: naming conventions, error handling, design principles, and common patterns (validation, audit).

Quick Reference

```

-- PROCEDURE syntax
CREATE OR REPLACE PROCEDURE proc_name
(
    p_in_param      IN      datatype,
    p_out_param     OUT     datatype,
    p_inout_param   IN OUT  datatype,
    p_default_param IN      datatype DEFAULT value
)
IS
    v_local_var datatype;
BEGIN
    -- statements
EXCEPTION
    WHEN exception THEN
        -- error handling
END proc_name;
/

-- Call a procedure
EXEC proc_name(val1, val2, val3);
-- or
BEGIN
    proc_name(param1 => val1, param2 => val2);
END;

-- FUNCTION syntax
CREATE OR REPLACE FUNCTION func_name
(
    p_param1 IN datatype,
    p_param2 IN datatype DEFAULT value

```

```

)
RETURN return_type
IS
    v_local_var datatype;
BEGIN
    -- statements
    RETURN value;
EXCEPTION
    WHEN exception THEN
        RETURN error_value;
END func_name;
/

-- Call a function
v_result := func_name(value1, value2);
-- or in SQL
SELECT func_name(column) FROM table;

```

Procedure vs Function Decision Guide

Choose a Procedure When	Choose a Function When
Performing an action (update, delete, send)	Calculating or retrieving a value
You need multiple output values (use OUT parameters)	You need exactly one return value
Modifying database data	Read-only operations
It is a complex business process	It is a simple calculation or lookup
It won't be used in SQL queries	You want to use it in SELECT/WHERE/HAVING

Practice Exercises

1. Simple Procedure: Create a procedure that displays all students enrolled in a given course. Pass the course ID as a parameter.
2. Simple Function: Create a function that returns the full name (first + last) of a student given their ID.
3. OUT Parameter: Create a procedure that accepts a course ID and returns (via OUT parameters) the course name, instructor, number of enrolled students, and average score.
4. IN OUT Parameter: Create a procedure that accepts a salary (IN OUT) and increases it by a given percentage. The modified salary should be returned in the same parameter.
5. Function with Calculation: Create a function that calculates a student's age from date_of_birth. Return the result as a whole number of years.
6. Default Parameters: Create a procedure to enroll a student with these parameters: student_id (required), course_id (required), enrollment_date (default SYSDATE), status (default 'Active').
7. Validation Function: Create a function that checks whether a GPA value is valid (between 0 and 4, with at most two decimal places).
8. Named Notation: Create a procedure with 5 parameters (use defaults for some). Call it using named notation, skipping some of the optional parameters.
9. Function in SQL: Create a function that returns how many years an employee has been with the company based on hire_date. Use it in a SELECT query to list all employees with their years of service.
10. Complete Application: Create a system with a function get_student_average(student_id) that returns the student's average score, a function get_performance_level(student_id) that returns 'High', 'Medium', or 'Low' based on that average, and a procedure update_gpa(student_id) that

recalculates the GPA from the average score (for example, average / 25). Test all three together in a block that processes several students.

Common Errors and Solutions

Error	Cause	Solution
PLS-00306: wrong number or types of arguments	Incorrect parameters when calling	Check that the parameter count and types match the declaration
PLS-00372: RETURN not allowed	Using RETURN with a value in a procedure	Use OUT parameters instead, or create a function
ORA-06502: numeric or value error	Data type mismatch or size issue	Make sure OUT parameter variables are large enough
Function returned without value	Missing RETURN in some code path	Make sure every path has a RETURN, including the EXCEPTION section
Cannot perform DML inside a query	Function modifies data but is used in SELECT	Remove the DML, or call the function only from PL/SQL

Chapter 5: Packages: Organizing Related Code

In this chapter you will learn:

- What packages are and why they are important
- Package specification vs package body
- How to create and use packages
- Public vs private elements
- Package variables and initialization
- Overloading procedures and functions
- Good practice for package design

5.1 Understanding Packages

What is a Package?

A package is a container that groups related procedures, functions, variables, cursors, and other PL/SQL elements together. Think of it as a toolbox that holds related tools.

Real-World Analogy

Imagine organizing the tools in your garage:

- ✓ Without packages: tools are scattered everywhere; a hammer here, a screwdriver there, nails somewhere else.
- ✓ With packages: a toolbox labelled "Carpentry Tools" holds the hammer, saw, nails and screws all in one place.

In the same way, a student records package keeps all the student-related procedures and functions together.

Why Use Packages?

Benefit	Description	Example
Organization	Group related code logically	All salary functions in one package
Encapsulation	Hide implementation details	Public interface, private helper functions
Performance	Loaded into memory as one unit	Faster than calling many separate procedures
Easier maintenance	All related code in one place	Change the salary logic in one package
Overloading	Same name, different parameters	Multiple versions of calculate_bonus

5.2 Package Structure: Specification and Body

Every package has two parts.

1. Package Specification (the Interface)

This is the public part: what others can see and use.

```

CREATE OR REPLACE PACKAGE package_name IS
    -- Public declarations
    -- Procedure and function signatures (headers only)
    -- Variables, constants, cursors, types

    PROCEDURE proc_name(parameters);
    FUNCTION func_name(parameters) RETURN datatype;
END package_name;
/

```

2. Package Body (the Implementation)

This is the private part: the actual code.

```

CREATE OR REPLACE PACKAGE BODY package_name IS
    -- Private declarations (optional)
    -- Private procedures/functions
    -- Implementation of the public procedures/functions

    PROCEDURE proc_name(parameters) IS
    BEGIN
        -- actual code here
    END;

    FUNCTION func_name(parameters) RETURN datatype IS
    BEGIN
        -- actual code here
        RETURN value;
    END;
END package_name;
/

```

Important Concepts

- Specification = Menu: it shows what is available.
- Body = Kitchen: it is where the actual work happens.
- Public: declared in the specification; anyone can use it.
- Private: exists only in the body; for internal use only.

5.3 Creating Your First Package

Example 1: Simple Employee Package

Step 1: Create the package specification

```

CREATE OR REPLACE PACKAGE emp_pkg IS
    -- Public function: get employee full name
    FUNCTION get_full_name(p_emp_id IN NUMBER) RETURN VARCHAR2;

    -- Public procedure: give a raise to an employee
    PROCEDURE give_raise(p_emp_id IN NUMBER, p_percent IN NUMBER);

    -- Public function: calculate monthly salary (salaries are stored as annual amounts)
    FUNCTION get_monthly_salary(p_emp_id IN NUMBER) RETURN NUMBER;
END emp_pkg;
/

```

Step 2: Create the package body

```

CREATE OR REPLACE PACKAGE BODY emp_pkg IS

    -- Implementation of get_full_name
    FUNCTION get_full_name(p_emp_id IN NUMBER) RETURN VARCHAR2 IS

```

```

        v_name VARCHAR2(100);
BEGIN
    SELECT first_name || ' ' || last_name
    INTO v_name
    FROM employees
    WHERE employee_id = p_emp_id;

    RETURN v_name;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Unknown Employee';
END get_full_name;

-- Implementation of give_raise
PROCEDURE give_raise(p_emp_id IN NUMBER, p_percent IN NUMBER) IS
    v_name VARCHAR2(100);
BEGIN
    v_name := get_full_name(p_emp_id);    -- Call a package function

    UPDATE employees
    SET salary = salary * (1 + p_percent / 100)
    WHERE employee_id = p_emp_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Raise given to: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Percentage: ' || p_percent || '%');
END give_raise;

-- Implementation of get_monthly_salary
FUNCTION get_monthly_salary(p_emp_id IN NUMBER) RETURN NUMBER IS
    v_annual_salary NUMBER;
BEGIN
    SELECT salary INTO v_annual_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    RETURN ROUND(v_annual_salary / 12, 2);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END get_monthly_salary;

END emp_pkg;
/

```

Step 3: Use the package

```

BEGIN
    -- Call a package function
    DBMS_OUTPUT.PUT_LINE('Employee: ' || emp_pkg.get_full_name(1));

    -- Call a package procedure
    emp_pkg.give_raise(1, 10);

    -- Call another package function
    DBMS_OUTPUT.PUT_LINE('Monthly Salary: $' || emp_pkg.get_monthly_salary(1));
END;
/

```

5.4 Public vs Private Elements

Example 2: Package with Private Helper Functions

Package specification (public interface)

```
CREATE OR REPLACE PACKAGE salary_pkg IS
  -- Public procedure
  PROCEDURE process_annual_review(p_emp_id IN NUMBER, p_rating IN NUMBER);

  -- Public function
  FUNCTION get_salary_grade(p_emp_id IN NUMBER) RETURN VARCHAR2;
END salary_pkg;
/
```

Package body (public + private)

```
CREATE OR REPLACE PACKAGE BODY salary_pkg IS

  -- PRIVATE function (not in the specification)
  FUNCTION calculate_raise_percent(p_rating IN NUMBER) RETURN NUMBER IS
  BEGIN
    CASE p_rating
      WHEN 5 THEN RETURN 20;    -- Excellent
      WHEN 4 THEN RETURN 15;    -- Very Good
      WHEN 3 THEN RETURN 10;    -- Good
      WHEN 2 THEN RETURN 5;     -- Fair
      ELSE RETURN 0;            -- Poor
    END CASE;
  END calculate_raise_percent;

  -- PRIVATE procedure (not in the specification)
  PROCEDURE log_salary_change(p_emp_id IN NUMBER,
                             p_old_sal IN NUMBER,
                             p_new_sal IN NUMBER) IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Employee ' || p_emp_id ||
      ' salary changed: $' || p_old_sal ||
      ' -> $' || p_new_sal);
  END log_salary_change;

  -- PUBLIC procedure implementation
  PROCEDURE process_annual_review(p_emp_id IN NUMBER, p_rating IN NUMBER) IS
    v_raise_pct NUMBER;
    v_old_salary NUMBER;
    v_new_salary NUMBER;
  BEGIN
    -- Use the private function
    v_raise_pct := calculate_raise_percent(p_rating);

    SELECT salary INTO v_old_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    v_new_salary := v_old_salary * (1 + v_raise_pct / 100);

    UPDATE employees
    SET salary = v_new_salary
    WHERE employee_id = p_emp_id;

    -- Use the private procedure
    log_salary_change(p_emp_id, v_old_salary, v_new_salary);

    COMMIT;
  END process_annual_review;

  -- PUBLIC function implementation
  FUNCTION get_salary_grade(p_emp_id IN NUMBER) RETURN VARCHAR2 IS
```

```

        v_salary NUMBER;
BEGIN
    SELECT salary INTO v_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    IF v_salary >= 95000 THEN RETURN 'Executive';
    ELIF v_salary >= 80000 THEN RETURN 'Senior';
    ELIF v_salary >= 65000 THEN RETURN 'Mid-Level';
    ELIF v_salary >= 50000 THEN RETURN 'Junior';
    ELSE RETURN 'Entry';
    END IF;
END get_salary_grade;

END salary_pkg;
/

```

Using the package

```

BEGIN
    -- Public elements can be called
    salary_pkg.process_annual_review(1, 4);
    DBMS_OUTPUT.PUT_LINE('Grade: ' || salary_pkg.get_salary_grade(1));

    -- Private elements CANNOT be called; this would give an error:
    -- salary_pkg.calculate_raise_percent(4);
END;
/

```

5.5 Package Variables and State

Package variables keep their values throughout your session. They behave like global variables for your package.

Example 3: Package with State Variables

```

CREATE OR REPLACE PACKAGE stats_pkg IS
    -- Public procedure to track salary changes
    PROCEDURE track_salary_change(p_emp_id IN NUMBER,
                                   p_old_sal IN NUMBER,
                                   p_new_sal IN NUMBER);

    -- Public functions to get statistics
    FUNCTION get_total_changes RETURN NUMBER;
    FUNCTION get_total_increase RETURN NUMBER;

    -- Public procedure to reset statistics
    PROCEDURE reset_stats;
END stats_pkg;
/

CREATE OR REPLACE PACKAGE BODY stats_pkg IS
    -- Package variables (keep their values during the session)
    g_change_count NUMBER := 0;
    g_total_increase NUMBER := 0;

    PROCEDURE track_salary_change(p_emp_id IN NUMBER,
                                   p_old_sal IN NUMBER,
                                   p_new_sal IN NUMBER) IS
    BEGIN
        g_change_count := g_change_count + 1;
        g_total_increase := g_total_increase + (p_new_sal - p_old_sal);

        DBMS_OUTPUT.PUT_LINE('Employee ' || p_emp_id ||
                              ' salary change tracked: $' || (p_new_sal - p_old_sal));
    END track_salary_change;

```

```

FUNCTION get_total_changes RETURN NUMBER IS
BEGIN
    RETURN g_change_count;
END get_total_changes;

FUNCTION get_total_increase RETURN NUMBER IS
BEGIN
    RETURN g_total_increase;
END get_total_increase;

PROCEDURE reset_stats IS
BEGIN
    g_change_count := 0;
    g_total_increase := 0;
    DBMS_OUTPUT.PUT_LINE('Statistics reset');
END reset_stats;
END stats_pkg;
/

-- Using package state
BEGIN
    stats_pkg.track_salary_change(1, 85000, 93500);
    stats_pkg.track_salary_change(2, 75000, 82500);
    stats_pkg.track_salary_change(3, 65000, 71500);

    DBMS_OUTPUT.PUT_LINE('Total changes: ' || stats_pkg.get_total_changes);
    DBMS_OUTPUT.PUT_LINE('Total increase: $' || stats_pkg.get_total_increase);

    stats_pkg.reset_stats;
END;
/

```

5.6 Package Initialization

You can add initialization code that runs once, when the package is first loaded into memory.

Example 4: Package with an Initialization Block

```

CREATE OR REPLACE PACKAGE config_pkg IS
    FUNCTION get_tax_rate RETURN NUMBER;
    FUNCTION get_max_bonus_pct RETURN NUMBER;
    PROCEDURE display_config;
END config_pkg;
/

CREATE OR REPLACE PACKAGE BODY config_pkg IS
    -- Package variables
    g_tax_rate          NUMBER;
    g_max_bonus_pct     NUMBER;
    g_initialized_date  DATE;

    FUNCTION get_tax_rate RETURN NUMBER IS
    BEGIN
        RETURN g_tax_rate;
    END;

    FUNCTION get_max_bonus_pct RETURN NUMBER IS
    BEGIN
        RETURN g_max_bonus_pct;
    END;

    PROCEDURE display_config IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('=== Configuration ===');
        DBMS_OUTPUT.PUT_LINE('Tax Rate: ' || g_tax_rate || '%');
    END;

```

```

        DBMS_OUTPUT.PUT_LINE('Max Bonus: ' || g_max_bonus_pct || '%');
        DBMS_OUTPUT.PUT_LINE('Initialized: ' || g_initialized_date);
    END;

-- INITIALIZATION SECTION (runs once when the package is first loaded)
BEGIN
    g_tax_rate := 15;
    g_max_bonus_pct := 20;
    g_initialized_date := SYSDATE;
    DBMS_OUTPUT.PUT_LINE('Package config_pkg initialized!');
END config_pkg;
/

```

5.7 Overloading in Packages

Overloading means having several procedures or functions with the same name but different parameters. Oracle decides which version to call from the number and types of the arguments.

Example 5: Overloaded Functions

```

CREATE OR REPLACE PACKAGE calc_pkg IS
    -- Version 1: by salary
    FUNCTION calculate_bonus(p_salary IN NUMBER) RETURN NUMBER;

    -- Version 2: by salary and rating
    FUNCTION calculate_bonus(p_salary IN NUMBER, p_rating IN NUMBER) RETURN NUMBER;

    -- Version 3: by employee last name
    FUNCTION calculate_bonus(p_last_name IN VARCHAR2) RETURN NUMBER;
END calc_pkg;
/

CREATE OR REPLACE PACKAGE BODY calc_pkg IS

    -- Version 1: by salary
    FUNCTION calculate_bonus(p_salary IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN p_salary * 0.10;    -- 10% of salary
    END calculate_bonus;

    -- Version 2: by salary and rating
    FUNCTION calculate_bonus(p_salary IN NUMBER, p_rating IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN p_salary * (p_rating * 0.05);    -- 5% per rating point
    END calculate_bonus;

    -- Version 3: by employee last name
    FUNCTION calculate_bonus(p_last_name IN VARCHAR2) RETURN NUMBER IS
        v_salary NUMBER;
    BEGIN
        SELECT salary INTO v_salary
        FROM employees
        WHERE last_name = p_last_name
        AND ROWNUM = 1;

        RETURN v_salary * 0.10;    -- 10% of salary
    END calculate_bonus;

END calc_pkg;
/

-- Using the overloaded functions
DECLARE
    v_bonus NUMBER;
BEGIN
    -- Version 1 (one number)

```

```

v_bonus := calc_pkg.calculate_bonus(60000);
DBMS_OUTPUT.PUT_LINE('Bonus (by salary): $' || v_bonus);

-- Version 2 (two numbers)
v_bonus := calc_pkg.calculate_bonus(60000, 4);
DBMS_OUTPUT.PUT_LINE('Bonus (by salary and rating): $' || v_bonus);

-- Version 3 (text)
v_bonus := calc_pkg.calculate_bonus('Manager'); -- John Manager
DBMS_OUTPUT.PUT_LINE('Bonus (by last name): $' || v_bonus);
END;
/

```

Note: overloaded versions must differ in the number, order, or data type family of their parameters. Two versions that each take a single NUMBER (for example one called p_emp_id and one called p_salary) cannot be told apart, and calling them produces error PLS-00307 (too many declarations match this call).

5.8 Complete Student Records Package

Example 6: Comprehensive Student Records Package

This package brings together the everyday student-records operations on the STUDENTS, COURSES and ENROLLMENTS tables.

Package specification

```

CREATE OR REPLACE PACKAGE student_mgmt_pkg IS
-- Student information
FUNCTION get_student_details(p_student_id IN NUMBER) RETURN VARCHAR2;
FUNCTION get_average_score(p_student_id IN NUMBER) RETURN NUMBER;

-- Enrollment operations
PROCEDURE enroll_student(p_student_id IN NUMBER, p_course_id IN NUMBER);
PROCEDURE record_score(p_enrollment_id IN NUMBER, p_score IN NUMBER);

-- Academic records
PROCEDURE update_gpa(p_student_id IN NUMBER);

-- Reporting
PROCEDURE generate_course_report(p_course_id IN NUMBER);
END student_mgmt_pkg;
/

```

Package body

```

CREATE OR REPLACE PACKAGE BODY student_mgmt_pkg IS

-- Private constant
c_pass_mark CONSTANT NUMBER := 50;

-- Private function: convert a score to a letter grade
FUNCTION score_to_grade(p_score IN NUMBER) RETURN VARCHAR2 IS
BEGIN
    IF p_score >= 90 THEN RETURN 'A';
    ELSIF p_score >= 80 THEN RETURN 'B';
    ELSIF p_score >= 70 THEN RETURN 'C';
    ELSIF p_score >= c_pass_mark THEN RETURN 'D';
    ELSE RETURN 'F';
    END IF;
END score_to_grade;

-- Private function: does the course still have a free seat?
FUNCTION course_has_seat(p_course_id IN NUMBER) RETURN BOOLEAN IS

```

```

        v_max          NUMBER;
        v_enrolled NUMBER;
BEGIN
    SELECT max_students INTO v_max
    FROM courses
    WHERE course_id = p_course_id;

    SELECT COUNT(*) INTO v_enrolled
    FROM enrollments
    WHERE course_id = p_course_id
    AND status = 'Active';

    RETURN v_enrolled < v_max;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END course_has_seat;

-- Public function: get student details
FUNCTION get_student_details(p_student_id IN NUMBER) RETURN VARCHAR2 IS
    v_details VARCHAR2(500);
BEGIN
    SELECT first_name || ' ' || last_name ||
           ' - ' || email ||
           ' - GPA: ' || gpa ||
           ' - Status: ' || status
    INTO v_details
    FROM students
    WHERE student_id = p_student_id;

    RETURN v_details;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Student not found';
END get_student_details;

-- Public function: average of the student's graded courses
FUNCTION get_average_score(p_student_id IN NUMBER) RETURN NUMBER IS
    v_avg NUMBER;
BEGIN
    SELECT AVG(score) INTO v_avg
    FROM enrollments
    WHERE student_id = p_student_id
    AND score IS NOT NULL;

    RETURN NVL(ROUND(v_avg, 2), 0);
END get_average_score;

-- Public procedure: enroll a student in a course
PROCEDURE enroll_student(p_student_id IN NUMBER, p_course_id IN NUMBER) IS
BEGIN
    IF NOT course_has_seat(p_course_id) THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Course ' || p_course_id || ' is full or does not exist');
    END IF;

    INSERT INTO enrollments (enrollment_id, student_id, course_id,
                           enrollment_date, status)
    VALUES (enrollment_seq.NEXTVAL, p_student_id, p_course_id,
            SYSDATE, 'Active');

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Student ' || p_student_id ||
        ' enrolled in course ' || p_course_id);
END enroll_student;

-- Public procedure: record a score and its letter grade
PROCEDURE record_score(p_enrollment_id IN NUMBER, p_score IN NUMBER) IS
    v_grade VARCHAR2(2);

```

```

BEGIN
    IF p_score NOT BETWEEN 0 AND 100 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Score must be between 0 and 100');
    END IF;

    -- A private function cannot be called inside a SQL statement,
    -- so the grade is worked out first and stored in a variable
    v_grade := score_to_grade(p_score);

    UPDATE enrollments
    SET score = p_score,
        grade = v_grade
    WHERE enrollment_id = p_enrollment_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Enrollment ' || p_enrollment_id || ' not found');
    END IF;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Enrollment ' || p_enrollment_id || ': score ' ||
        p_score || ' (' || v_grade || ')');
END record_score;

-- Public procedure: recalculate GPA from the average score
PROCEDURE update_gpa(p_student_id IN NUMBER) IS
    v_avg      NUMBER;
    v_new_gpa  NUMBER(3,2);
BEGIN
    v_avg := get_average_score(p_student_id);

    IF v_avg = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Student ' || p_student_id ||
            ' has no graded courses yet');
        RETURN;
    END IF;

    -- Simple rule for class: GPA = average score / 25, capped at 4.00
    v_new_gpa := LEAST(ROUND(v_avg / 25, 2), 4);

    UPDATE students
    SET gpa = v_new_gpa
    WHERE student_id = p_student_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Student ' || p_student_id || ': average ' ||
        v_avg || ', new GPA ' || v_new_gpa);
END update_gpa;

-- Public procedure: course report
PROCEDURE generate_course_report(p_course_id IN NUMBER) IS
    v_course_name  VARCHAR2(100);
    v_instructor   VARCHAR2(50);
    v_max_students  NUMBER;
    v_count        NUMBER;
    v_avg_score     NUMBER;
BEGIN
    SELECT c.course_name, c.instructor, c.max_students,
        COUNT(e.enrollment_id), AVG(e.score)
    INTO v_course_name, v_instructor, v_max_students, v_count, v_avg_score
    FROM courses c
    LEFT JOIN enrollments e ON c.course_id = e.course_id
    WHERE c.course_id = p_course_id
    GROUP BY c.course_name, c.instructor, c.max_students;

    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('COURSE REPORT: ' || v_course_name);
    DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Instructor: ' || v_instructor);

```

```

DBMS_OUTPUT.PUT_LINE('Enrolled: ' || v_count || ' of ' || v_max_students);
DBMS_OUTPUT.PUT_LINE('Average Score: ' ||
    NVL(TO_CHAR(ROUND(v_avg_score, 2)), 'No scores yet'));
DBMS_OUTPUT.PUT_LINE('=====');

IF v_count > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Student List:');
    FOR rec IN (SELECT s.first_name, s.last_name, e.score, e.grade
        FROM enrollments e
        JOIN students s ON e.student_id = s.student_id
        WHERE e.course_id = p_course_id
        ORDER BY e.score DESC NULLS LAST) LOOP
        DBMS_OUTPUT.PUT_LINE(' ' || rec.first_name || ' ' || rec.last_name ||
            ' - ' || NVL(TO_CHAR(rec.score), 'Not graded') ||
            ' ' || rec.grade);
    END LOOP;
END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Course ' || p_course_id || ' not found');
END generate_course_report;

END student_mgmt_pkg;
/

```

Using the complete package

```

BEGIN
    -- Student information
    DBMS_OUTPUT.PUT_LINE(student_mgmt_pkg.get_student_details(1001));
    DBMS_OUTPUT.PUT_LINE('Average score: ' ||
        student_mgmt_pkg.get_average_score(1001));

    -- Enroll student 1012 in Data Structures (course 104)
    student_mgmt_pkg.enroll_student(1012, 104);

    -- Record the missing score for enrollment 3 (student 1001, course 103)
    student_mgmt_pkg.record_score(3, 87);

    -- Recalculate student 1001's GPA
    student_mgmt_pkg.update_gpa(1001);

    -- Course report
    DBMS_OUTPUT.PUT_LINE('');
    student_mgmt_pkg.generate_course_report(103);
END;
/

```

5.9 Managing Packages

Viewing Package Information

```
-- List all packages
SELECT object_name, object_type, status
FROM user_objects
WHERE object_type IN ('PACKAGE', 'PACKAGE BODY')
ORDER BY object_name;

-- View the package specification source
SELECT text
FROM user_source
WHERE name = 'STUDENT_MGMT_PKG' AND type = 'PACKAGE'
ORDER BY line;

-- View the package body source
SELECT text
FROM user_source
WHERE name = 'STUDENT_MGMT_PKG' AND type = 'PACKAGE BODY'
ORDER BY line;

-- Check for compilation errors
SELECT line, position, text
FROM user_errors
WHERE name = 'STUDENT_MGMT_PKG'
ORDER BY line, position;
```

Dropping and Recompiling Packages

```
-- Drop the entire package (specification and body)
DROP PACKAGE student_mgmt_pkg;

-- Drop only the package body (keeps the specification)
DROP PACKAGE BODY student_mgmt_pkg;

-- Recompile the package
ALTER PACKAGE student_mgmt_pkg COMPILE;
ALTER PACKAGE student_mgmt_pkg COMPILE BODY;
```

5.10 Package Design Guidelines

Organization:

- ✓ Group related functionality together.
- ✓ Use one package per business area (student records, staff, finance).
- ✓ Keep packages focused; don't make them too large.
- ✓ Use meaningful package names (student_mgmt_pkg, not pkg1).

Encapsulation:

- ✓ Put only what is necessary in the specification (public).
- ✓ Keep implementation details private (in the body only).
- ✓ Use private helper functions for internal logic.
- ✓ Expose clean, simple interfaces to users.

Naming conventions:

- ✓ Package names: end with _pkg (student_mgmt_pkg)
- ✓ Global variables: prefix with g_ (g_tax_rate)
- ✓ Constants: prefix with c_ (c_max_salary)

- ✓ Parameters: prefix with p_ (p_emp_id)
- ✓ Local variables: prefix with v_ (v_total)

Performance:

- ✓ Initialize package variables wisely.
- ✓ Avoid unnecessary package state.
- ✓ Use package constants for fixed values.
- ✓ Consider package-level cursors for frequently used queries.

5.11 Common Package Patterns

Example 7: Constants Package

```
-- Package for application constants
CREATE OR REPLACE PACKAGE constants_pkg IS
    -- Academic rules
    c_min_gpa          CONSTANT NUMBER := 0;
    c_max_gpa          CONSTANT NUMBER := 4;
    c_deans_list_gpa   CONSTANT NUMBER := 3.5;
    c_pass_mark        CONSTANT NUMBER := 50;

    -- Salary limits
    c_min_salary        CONSTANT NUMBER := 30000;
    c_max_salary        CONSTANT NUMBER := 150000;

    -- Bonus percentages
    c_excellent_bonus_pct CONSTANT NUMBER := 20;
    c_good_bonus_pct      CONSTANT NUMBER := 15;
    c_average_bonus_pct   CONSTANT NUMBER := 10;

    -- Status codes (as used in STUDENTS and ENROLLMENTS)
    c_status_active      CONSTANT VARCHAR2(20) := 'Active';
    c_status_inactive    CONSTANT VARCHAR2(20) := 'Inactive';
    c_status_completed   CONSTANT VARCHAR2(20) := 'Completed';
END constants_pkg;
/

-- Using the constants
DECLARE
    v_gpa      students.gpa%TYPE;
    v_status   students.status%TYPE;
BEGIN
    SELECT gpa, status INTO v_gpa, v_status
    FROM students
    WHERE student_id = 1005;

    IF v_status = constants_pkg.c_status_active
       AND v_gpa >= constants_pkg.c_deans_list_gpa THEN
        DBMS_OUTPUT.PUT_LINE('Student 1005 is on the Dean''s List');
    END IF;
END;
/
```

Example 8: Utility Package

```
CREATE OR REPLACE PACKAGE util_pkg IS
    -- String utilities
    FUNCTION capitalize(p_string IN VARCHAR2) RETURN VARCHAR2;
    FUNCTION proper_case(p_string IN VARCHAR2) RETURN VARCHAR2;

    -- Date utilities
    FUNCTION get_age(p_birth_date IN DATE) RETURN NUMBER;
    FUNCTION is_business_day(p_date IN DATE) RETURN BOOLEAN;
END;
```

```

-- Number utilities
FUNCTION format_currency(p_amount IN NUMBER) RETURN VARCHAR2;
FUNCTION round_to_nearest_hundred(p_amount IN NUMBER) RETURN NUMBER;
END util_pkg;
/

CREATE OR REPLACE PACKAGE BODY util_pkg IS

    FUNCTION capitalize(p_string IN VARCHAR2) RETURN VARCHAR2 IS
    BEGIN
        RETURN UPPER(SUBSTR(p_string, 1, 1)) || LOWER(SUBSTR(p_string, 2));
    END capitalize;

    FUNCTION proper_case(p_string IN VARCHAR2) RETURN VARCHAR2 IS
    BEGIN
        RETURN INITCAP(p_string);
    END proper_case;

    FUNCTION get_age(p_birth_date IN DATE) RETURN NUMBER IS
    BEGIN
        RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, p_birth_date) / 12);
    END get_age;

    FUNCTION is_business_day(p_date IN DATE) RETURN BOOLEAN IS
        v_day VARCHAR2(10);
    BEGIN
        v_day := TO_CHAR(p_date, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH');
        RETURN v_day NOT IN ('SAT', 'SUN');
    END is_business_day;

    FUNCTION format_currency(p_amount IN NUMBER) RETURN VARCHAR2 IS
    BEGIN
        RETURN '$' || TRIM(TO_CHAR(p_amount, '999,999,999.99'));
    END format_currency;

    FUNCTION round_to_nearest_hundred(p_amount IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN ROUND(p_amount, -2);
    END round_to_nearest_hundred;

END util_pkg;
/

```

5.12 Package Dependencies

When a package references other database objects (tables, views, other packages), it depends on them:

- ✓ If a table structure changes, the packages that use it become INVALID.
- ✓ Invalid packages are recompiled automatically the next time they are called.
- ✓ You can also recompile manually with `ALTER PACKAGE package_name COMPILE;`

```

-- View package dependencies
SELECT name, type, referenced_name, referenced_type
FROM user_dependencies
WHERE name = 'STUDENT_MGMT_PKG'
ORDER BY referenced_name;

-- Find what depends on a table
SELECT name, type
FROM user_dependencies
WHERE referenced_name = 'ENROLLMENTS'
AND referenced_type = 'TABLE';

```

Summary of Chapter 5

1. Package fundamentals: what packages are and why they are essential; the specification (interface) vs the body (implementation); the benefits of organization, encapsulation, and performance.
2. Creating packages: the two-step process (specification, then body); public elements (in the specification) vs private elements (body only); how to call package procedures and functions.
3. Advanced features: package variables and state, initialization blocks, overloading, and private helper functions.
4. Design guidelines: naming conventions (g_, c_, p_, v_), organizing code logically, encapsulation, and common patterns (constants, utilities).

Quick Reference

```
-- PACKAGE SPECIFICATION
CREATE OR REPLACE PACKAGE package_name IS
    -- Public constants
    c_constant_name CONSTANT datatype := value;

    -- Public variables
    g_variable_name datatype;

    -- Public procedure
    PROCEDURE proc_name(params);

    -- Public function
    FUNCTION func_name(params) RETURN datatype;
END package_name;
/

-- PACKAGE BODY
CREATE OR REPLACE PACKAGE BODY package_name IS
    -- Private variables/constants
    g_private_var datatype;

    -- Private procedures/functions (not in the spec)
    FUNCTION private_func RETURN datatype IS
    BEGIN
        RETURN value;
    END;

    -- Public procedure implementation
    PROCEDURE proc_name(params) IS
    BEGIN
        -- code
    END;

    -- Public function implementation
    FUNCTION func_name(params) RETURN datatype IS
    BEGIN
        RETURN value;
    END;

    -- Initialization (optional)
    BEGIN
        g_private_var := initial_value;
    END package_name;
/

-- USAGE
BEGIN
    package_name.proc_name(values);
    v_result := package_name.func_name(values);
END;
```

Package Structure Comparison

Aspect	Specification	Body
Purpose	Defines the interface (what is available)	Implements the functionality (how it works)
Visibility	Public; visible to all	Can have public and private elements
Required?	Yes	Yes, if the specification declares procedures or functions
Effect of changes	Changes invalidate dependent objects	Can change without affecting dependents
Contains	Headers only	The complete implementation

Practice Exercises

- Simple Package: Create a package called `math_pkg` with functions to:
 - Calculate a percentage (part, whole)
 - Calculate the average of 3 numbers
 - Round to 2 decimal places
- Public vs Private: Create a package with:
 - Public procedure: `calculate_net_salary(emp_id)`
 - Private function: `get_tax_rate(salary)`, which returns the tax percentage
 - Private function: `get_deductions(salary)`, which returns the total deductions
- Package State: Create a counter package that:
 - Tracks how many times its procedures are called
 - Has a reset procedure
 - Has a `get_count` function
- Overloading: Create a search package with an overloaded `find_employee`:
 - By `employee_id`
 - By last name
 - By department and `job_title`
- Initialization: Create a config package that initializes company settings:
 - Default tax rate
 - Company name
 - Fiscal year start date
- Constants Package: Create `hr_constants_pkg` with:
 - Minimum and maximum salary constants
 - Standard work hours per week
 - Vacation days per year
- Utility Package: Create `string_utils_pkg` with:
 - A function to remove spaces
 - A function to reverse a string
 - A function to count characters

8. Department Package: Create dept_mgmt_pkg with:
 - a) Function: get_department_size(dept_id)
 - b) Procedure: transfer_all_employees(from_dept, to_dept)
 - c) Function: get_department_budget(dept_id)
9. Validation Package: Create validation_pkg with:
 - a) Function: is_valid_email(email)
 - b) Function: is_valid_phone(phone)
 - c) Function: is_valid_gpa(gpa)
10. Complete Staff System: Create a comprehensive staff_system_pkg for the EMPLOYEES and DEPARTMENTS tables with:
 - a) All employee CRUD operations (Create, Read, Update, Delete)
 - b) Salary management (raises, bonuses)
 - c) Department transfers
 - d) Reporting functions
 - e) Validation functions
 - f) Private helper functions where appropriate

Common Errors and Solutions

Error	Cause	Solution
PLS-00304: cannot compile body without its specification	Creating the body before the specification	Create the specification first, then the body
PLS-00323: subprogram declared in the specification must be defined in the body	Body doesn't match the specification	Make sure parameters and return types match exactly
PLS-00302: component must be declared	Calling a private element from outside the package	Add it to the specification, or call it from within the package
PLS-00307: too many declarations match this call	Overloaded versions that cannot be told apart	Make the parameter lists differ in number, order, or type family
Package has INVALID status	A dependency changed or there is a compilation error	Check user_errors and recompile the package
ORA-04068: existing state of packages has been discarded	The package was recompiled and its state was lost	Re-initialize package variables if needed

Chapter 6: Exception Handling: Managing Errors Gracefully

In this chapter you will learn:

- ✓ What exceptions are and why they occur
- ✓ Types of exceptions (predefined, user-defined, PRAGMA EXCEPTION_INIT)
- ✓ How to handle exceptions properly
- ✓ How to create and raise custom exceptions
- ✓ Exception propagation and scope
- ✓ Good practice for error handling

6.1 Understanding Exceptions

What is an Exception?

An exception is an error that occurs while a program is running. Instead of letting your program crash, PL/SQL allows you to catch these errors and handle them gracefully.

Real-World Analogy

Think of exceptions like emergencies in daily life:

- Without handling: the car breaks down and you are stuck on the highway.
- With handling: the car breaks down, you call roadside assistance, and the problem is resolved.

The same happens in programming:

- ✓ Without handling: an error occurs, the program crashes, and the user sees a cryptic error.
- ✓ With handling: an error occurs, your code handles it, and the user sees a friendly message.

Why Handle Exceptions?

What Happens Without Exception Handling

```
-- This will fail if the employee doesn't exist
DECLARE
    v_name    VARCHAR2(100);
    v_salary  NUMBER;
BEGIN
    SELECT first_name, salary
    INTO v_name, v_salary
    FROM employees
    WHERE employee_id = 9999;    -- This employee doesn't exist!

    DBMS_OUTPUT.PUT_LINE(v_name || ' earns $' || v_salary);
END;
/
-- Error: ORA-01403: no data found
-- The program stops and the user is confused.
```

With Exception Handling

```
DECLARE
    v_name    VARCHAR2(100);
    v_salary  NUMBER;
BEGIN
    SELECT first_name, salary
    INTO v_name, v_salary
    FROM employees
    WHERE employee_id = 9999;

    DBMS_OUTPUT.PUT_LINE(v_name || ' earns $' || v_salary);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found. Please check the ID.');
```

-- Output: Employee not found. Please check the ID.
-- The program finishes normally and the user gets a helpful message.

6.2 Exception Handling Syntax

Basic Exception Handling Structure

```
BEGIN
    -- Executable statements
    statements;
EXCEPTION
    WHEN exception_name1 THEN
        -- Handle a specific exception
        statements;
    WHEN exception_name2 THEN
        -- Handle another specific exception
        statements;
    WHEN OTHERS THEN
        -- Handle all other exceptions
        statements;
END;
```

Important Rules

- ✓ The EXCEPTION section is optional.
- ✓ If an exception occurs and is not handled, it propagates up to the enclosing block.
- ✓ WHEN OTHERS catches all exceptions not handled specifically, and must come last.
- ✓ Only one exception handler executes per exception.
- ✓ After handling, control goes to the end of the block, not back to where the error occurred.

6.3 Types of Exceptions

1. Predefined Oracle Exceptions

These are common exceptions that Oracle has already defined and named for you.

Exception Name	Oracle Error	When It Occurs	Example Cause
NO_DATA_FOUND	ORA-01403	SELECT INTO returns no rows	Student ID doesn't exist
TOO_MANY_ROWS	ORA-01422	SELECT INTO returns more than one row	Query matches several employees

Exception Name	Oracle Error	When It Occurs	Example Cause
VALUE_ERROR	ORA-06502	Data type conversion or size error	Number too large for the variable
ZERO_DIVIDE	ORA-01476	Division by zero	Calculating a percentage with a total of 0
DUP_VAL_ON_INDEX	ORA-00001	Unique constraint violated	Enrolling a student twice in the same course
INVALID_CURSOR	ORA-01001	Invalid cursor operation	Fetching from a closed cursor
LOGIN_DENIED	ORA-01017	Invalid username/password	Connecting with wrong credentials
PROGRAM_ERROR	ORA-06501	Internal PL/SQL error	A problem inside the PL/SQL engine

Example 1: Handling Multiple Predefined Exceptions

```

DECLARE
    v_emp_id NUMBER := &emp_id;    -- User input (substitution variable)
    v_name   VARCHAR2(50);
    v_salary NUMBER;
    v_bonus  NUMBER;
BEGIN
    -- Get employee data
    SELECT first_name, salary
    INTO v_name, v_salary
    FROM employees
    WHERE employee_id = v_emp_id;

    -- Calculate a 10% bonus
    v_bonus := v_salary * 0.10;

    -- Display the result
    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary: $' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Bonus: $' || v_bonus);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Employee ID ' || v_emp_id || ' not found');
        DBMS_OUTPUT.PUT_LINE('Please check the employee ID and try again');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Multiple employees found');
        DBMS_OUTPUT.PUT_LINE('This should not happen - check data integrity');
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Invalid data type or size');
        DBMS_OUTPUT.PUT_LINE('Please check input values');
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Division by zero detected');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Unexpected error occurred');
        DBMS_OUTPUT.PUT_LINE('Error code: ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Error message: ' || SQLERRM);
END;
/

```

6.4 Exception Information Functions

PL/SQL provides functions that give you information about the current exception.

Function	Returns	Usage
SQLCODE	The Oracle error number	Get the specific error code for logging
SQLERRM	The Oracle error message	Get a detailed description of the error
SQLERRM(code)	The message for a given error code	Get the message for any Oracle error

Example 2: Using SQLCODE and SQLERRM

```

CREATE OR REPLACE PROCEDURE safe_salary_update
(
    p_emp_id      IN NUMBER,
    p_new_salary  IN NUMBER
) IS
    v_old_salary  NUMBER;
BEGIN
    -- Get the current salary
    SELECT salary INTO v_old_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Update the salary
    UPDATE employees
    SET salary = p_new_salary
    WHERE employee_id = p_emp_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('SUCCESS: Salary updated from $' || v_old_salary ||
        ' to $' || p_new_salary);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Employee ' || p_emp_id || ' not found');
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Invalid salary amount: ' || p_new_salary);
    WHEN OTHERS THEN
        -- Log detailed error information
        DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERROR occurred:');
        DBMS_OUTPUT.PUT_LINE(' Employee ID: ' || p_emp_id);
        DBMS_OUTPUT.PUT_LINE(' New Salary: ' || p_new_salary);
        DBMS_OUTPUT.PUT_LINE(' Oracle Error Code: ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE(' Oracle Error Message: ' || SQLERRM);

        -- Always roll back on unexpected errors
        ROLLBACK;
END safe_salary_update;
/

-- Test the procedure
BEGIN
    safe_salary_update(1, 90000);           -- Valid update
    safe_salary_update(9999, 30000);        -- Invalid employee ID
    safe_salary_update(2, 999999999999);    -- Too large for NUMBER(10,2):
                                           -- caught by WHEN OTHERS (ORA-01438)
END;
/

```

6.5 User-Defined Exceptions

You can create your own exceptions for business-logic errors that Oracle does not detect automatically.

Creating User-Defined Exceptions

```
DECLARE
    -- Declare the exception
    exception_name EXCEPTION;
BEGIN
    -- Your code here
    IF some_condition THEN
        RAISE exception_name;    -- Trigger the exception manually
    END IF;
EXCEPTION
    WHEN exception_name THEN
        -- Handle your custom exception
        statements;
END;
```

Example 3: Business Logic Exceptions

This procedure enrolls a student in a course and enforces the university's rules.

```
CREATE OR REPLACE PROCEDURE enroll_student
(
    p_student_id IN NUMBER,
    p_course_id  IN NUMBER
) IS
    -- User-defined exceptions
    e_student_inactive EXCEPTION;
    e_course_full      EXCEPTION;
    e_gpa_too_low      EXCEPTION;

    v_status           students.status%TYPE;
    v_gpa              students.gpa%TYPE;
    v_course_code      courses.course_code%TYPE;
    v_max_students     courses.max_students%TYPE;
    v_enrolled         NUMBER;
BEGIN
    -- Get the student (raises NO_DATA_FOUND if the student doesn't exist)
    SELECT status, gpa INTO v_status, v_gpa
    FROM students
    WHERE student_id = p_student_id;

    IF v_status <> 'Active' THEN
        RAISE e_student_inactive;
    END IF;

    -- Get the course (raises NO_DATA_FOUND if the course doesn't exist)
    SELECT course_code, max_students INTO v_course_code, v_max_students
    FROM courses
    WHERE course_id = p_course_id;

    -- Business rule: the course must have a free seat
    SELECT COUNT(*) INTO v_enrolled
    FROM enrollments
    WHERE course_id = p_course_id
    AND status = 'Active';

    IF v_enrolled >= v_max_students THEN
        RAISE e_course_full;
    END IF;

    -- Business rule: 500-level courses need a GPA of at least 3.0
    IF v_course_code LIKE '%5__' AND v_gpa < 3.0 THEN
        RAISE e_gpa_too_low;
    END IF;
```

```

-- All checks passed: enroll the student
INSERT INTO enrollments (enrollment_id, student_id, course_id,
                        enrollment_date, status)
VALUES (enrollment_seq.NEXTVAL, p_student_id, p_course_id,
        SYSDATE, 'Active');

COMMIT;
DBMS_OUTPUT.PUT_LINE('SUCCESS: Student ' || p_student_id ||
                    ' enrolled in ' || v_course_code);

EXCEPTION
  WHEN e_student_inactive THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: Student ' || p_student_id ||
                        ' is not active (status: ' || v_status || ')');
  WHEN e_course_full THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || v_course_code || ' is full (' ||
                        v_enrolled || ' of ' || v_max_students || ' seats taken)');
  WHEN e_gpa_too_low THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || v_course_code ||
                        ' requires a GPA of 3.0 (student has ' || v_gpa || ')');
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: Student ' || p_student_id ||
                        ' or course ' || p_course_id || ' does not exist');
  WHEN DUP_VAL_ON_INDEX THEN
    -- UNIQUE(student_id, course_id) on ENROLLMENTS was violated
    DBMS_OUTPUT.PUT_LINE('ERROR: Student ' || p_student_id ||
                        ' is already enrolled in course ' || p_course_id);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: Unexpected error - ' || SQLERRM);
    ROLLBACK;
END enroll_student;
/

-- Test the procedure
BEGIN
  -- Valid enrollment: Jack (1010) into PL/SQL Programming (103)
  enroll_student(1010, 103);

  -- Already enrolled: Alice (1001) is in course 101
  enroll_student(1001, 101);

  -- GPA too low: Paul (1016, GPA 2.7) into Machine Learning (CS501)
  enroll_student(1016, 110);

  -- Student does not exist
  enroll_student(9999, 101);
END;
/

```

Output (with the original lab data):

```

SUCCESS: Student 1010 enrolled in CS202
ERROR: Student 1001 is already enrolled in course 101
ERROR: CS501 requires a GPA of 3.0 (student has 2.7)
ERROR: Student 9999 or course 101 does not exist

```

6.6 PRAGMA EXCEPTION_INIT

You can give your own names to specific Oracle error numbers using `PRAGMA EXCEPTION_INIT`. This lets you handle errors that have no predefined name.

Example 4: Using PRAGMA EXCEPTION_INIT

In the lab tables, `ENROLLMENTS.course_id` refers to `COURSES`, so a course that still has enrollments cannot be deleted.

```

DECLARE
    -- Declare exceptions and associate them with Oracle errors
    e_child_record_found EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_child_record_found, -2292);
    -- ORA-02292: integrity constraint violated - child record found

    e_parent_key_not_found EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_parent_key_not_found, -2291);
    -- ORA-02291: integrity constraint violated - parent key not found

    v_course_id NUMBER := 101;    -- Introduction to Programming
BEGIN
    -- Try to delete the course (it still has enrollments)
    DELETE FROM courses
    WHERE course_id = v_course_id;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Course ' || v_course_id || ' does not exist');
    ELSE
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Course ' || v_course_id || ' deleted successfully');
    END IF;

    -- This statement would raise ORA-02291, because student 9999 doesn't exist:
    -- INSERT INTO enrollments (enrollment_id, student_id, course_id)
    -- VALUES (enrollment_seq.NEXTVAL, 9999, 101);
EXCEPTION
    WHEN e_child_record_found THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Cannot delete course ' || v_course_id);
        DBMS_OUTPUT.PUT_LINE('Students are still enrolled in this course');
        DBMS_OUTPUT.PUT_LINE('Remove or move the enrollments first');
    WHEN e_parent_key_not_found THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: The student or course referred to does not exist');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
        ROLLBACK;
END;
/

```

Output:

```

ERROR: Cannot delete course 101
Students are still enrolled in this course
Remove or move the enrollments first

```

Note: a DELETE that matches no rows does not raise NO_DATA_FOUND. That exception is raised only by SELECT INTO, so SQL%ROWCOUNT is used above to detect a missing department.

6.7 Exception Propagation

When an exception occurs, it follows these rules:

- ✓ If it is handled in the current block, execution continues after that block's EXCEPTION section.
- ✓ If it is NOT handled in the current block, it propagates to the enclosing block.
- ✓ If it reaches the outermost block without being handled, the program terminates with an error.

Example 5: Exception Propagation

```
BEGIN    -- Outer block
    DBMS_OUTPUT.PUT_LINE('Outer block: Starting');

    DECLARE    -- Inner block
        v_salary NUMBER;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Inner block: Getting salary');

        SELECT salary INTO v_salary
        FROM employees
        WHERE employee_id = 9999;    -- This raises NO_DATA_FOUND

        DBMS_OUTPUT.PUT_LINE('Inner block: This line never executes');
    END;

    DBMS_OUTPUT.PUT_LINE('Outer block: After inner block');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Outer block: Caught NO_DATA_FOUND from inner block');
        DBMS_OUTPUT.PUT_LINE('Handling the error gracefully');
END;
/
```

Output:

```
Outer block: Starting
Inner block: Getting salary
Outer block: Caught NO_DATA_FOUND from inner block
Handling the error gracefully
```

The Same Example with the Inner Block Handling the Error

```
BEGIN    -- Outer block
    DBMS_OUTPUT.PUT_LINE('Outer block: Starting');

    DECLARE    -- Inner block
        v_salary NUMBER;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Inner block: Getting salary');

        SELECT salary INTO v_salary
        FROM employees
        WHERE employee_id = 9999;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Inner block: Handled NO_DATA_FOUND locally');
            -- The exception is handled here and does not propagate
    END;

    DBMS_OUTPUT.PUT_LINE('Outer block: Continuing normally');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Outer block: This won''t execute');
END;
/
```

Output:

```
Outer block: Starting
Inner block: Getting salary
Inner block: Handled NO_DATA_FOUND locally
Outer block: Continuing normally
```

6.8 Re-raising Exceptions

Sometimes you want to handle an exception partly (for example, log it) and then let it continue to propagate. Use RAISE on its own, without an exception name.

Example 6: Re-raising Exceptions

```
CREATE OR REPLACE PROCEDURE process_employee_raise
(
    p_emp_id      IN NUMBER,
    p_raise_pct   IN NUMBER
) IS
    v_old_salary  NUMBER;
    v_new_salary  NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Processing raise for employee ' || p_emp_id);

    SELECT salary INTO v_old_salary
    FROM employees
    WHERE employee_id = p_emp_id;

    v_new_salary := v_old_salary * (1 + p_raise_pct / 100);

    UPDATE employees
    SET salary = v_new_salary
    WHERE employee_id = p_emp_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Raise processed successfully');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Logging: Employee ' || p_emp_id || ' not found');
        -- Log the error, then re-raise it
        RAISE;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Logging: Unexpected error in salary update');
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        ROLLBACK;
        RAISE; -- Re-raise the exception
END process_employee_raise;
/

-- Calling the procedure with exception handling
BEGIN
    process_employee_raise(1, 10);      -- Valid
    process_employee_raise(9999, 10);  -- Invalid: raises an exception
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Main program: Employee not found - operation cancelled');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Main program: Error occurred - ' || SQLERRM);
END;
/
```

6.9 RAISE_APPLICATION_ERROR

This built-in procedure lets you raise your own error with a specific error number and message.

RAISE_APPLICATION_ERROR Syntax

```
RAISE_APPLICATION_ERROR(error_code, error_message [, keep_errors]);

-- error_code:      a number between -20000 and -20999
-- error_message:   your own message (max 2048 characters)
-- keep_errors:     TRUE to add to the error stack, FALSE to replace it (default)
```

Example 7: Using RAISE_APPLICATION_ERROR

```
CREATE OR REPLACE FUNCTION calculate_bonus
(
    p_emp_id          IN NUMBER,
    p_performance_rating IN NUMBER
)
RETURN NUMBER
IS
    v_salary          NUMBER;
    v_years_service   NUMBER;
    v_bonus           NUMBER;
BEGIN
    -- Validate the performance rating
    IF p_performance_rating NOT BETWEEN 1 AND 5 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Invalid performance rating: ' || p_performance_rating ||
            '. Must be between 1 and 5.');
```

```
    END IF;

    -- Get employee information
    SELECT salary,
        ROUND(MONTHS_BETWEEN(SYSDATE, hire_date) / 12, 1)
    INTO v_salary, v_years_service
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Business rule: no bonus for employees with less than 1 year of service
    IF v_years_service < 1 THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Employee ' || p_emp_id || ' not eligible for bonus. ' ||
            'Minimum 1 year service required (current: ' ||
            v_years_service || ' years).');
```

```
    END IF;

    -- Business rule: maximum salary for bonus consideration
    IF v_salary > 100000 THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Employee salary too high for bonus program. ' ||
            'Maximum eligible salary: $100,000 (current: $' || v_salary || ').');
```

```
    END IF;

    -- Calculate the bonus based on the rating
    v_bonus := v_salary * (p_performance_rating * 0.05);

    RETURN v_bonus;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004,
            'Employee ID ' || p_emp_id || ' not found in database.');
```

```
END calculate_bonus;
/

-- Test the function
DECLARE
    v_bonus NUMBER;
BEGIN
    -- Valid case
    v_bonus := calculate_bonus(5, 4);
    DBMS_OUTPUT.PUT_LINE('Bonus calculated: $' || v_bonus);

    -- Invalid rating
    v_bonus := calculate_bonus(5, 10);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error Code: ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Error Message: ' || SQLERRM);
END;
/
```

6.10 Exception Handling in Procedures and Functions

Example 8: Robust Employee Management Package

```
CREATE OR REPLACE PACKAGE emp_mgmt_pkg IS
    -- Salary limits used for validation
    c_min_salary CONSTANT NUMBER := 30000;
    c_max_salary CONSTANT NUMBER := 150000;

    -- Procedures with proper exception handling
    PROCEDURE hire_employee(p_first_name IN VARCHAR2, p_last_name IN VARCHAR2,
                           p_email IN VARCHAR2, p_job_title IN VARCHAR2,
                           p_salary IN NUMBER, p_department IN VARCHAR2);
    PROCEDURE fire_employee(p_emp_id IN NUMBER);
    PROCEDURE give_raise(p_emp_id IN NUMBER, p_percent IN NUMBER);

    -- Function with exception handling
    FUNCTION get_employee_info(p_emp_id IN NUMBER) RETURN VARCHAR2;
END emp_mgmt_pkg;
/

CREATE OR REPLACE PACKAGE BODY emp_mgmt_pkg IS

    -- Private procedure for logging errors
    PROCEDURE log_error(p_procedure IN VARCHAR2, p_error IN VARCHAR2) IS
    BEGIN
        -- In a real application, this would write to a log table
        DBMS_OUTPUT.PUT_LINE('LOG: ' || p_procedure || ' - ' || p_error);
    END log_error;

    -- Private function to validate a department name
    FUNCTION is_valid_department(p_department IN VARCHAR2) RETURN BOOLEAN IS
        v_count NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_count
        FROM departments
        WHERE department_name = p_department;

        RETURN v_count > 0;
    END is_valid_department;

    PROCEDURE hire_employee(p_first_name IN VARCHAR2, p_last_name IN VARCHAR2,
                           p_email IN VARCHAR2, p_job_title IN VARCHAR2,
                           p_salary IN NUMBER, p_department IN VARCHAR2) IS
        v_new_emp_id NUMBER;
    BEGIN
        -- Validate the department
        IF NOT is_valid_department(p_department) THEN
            RAISE_APPLICATION_ERROR(-20010, 'Invalid department: ' || p_department);
        END IF;

        -- Validate the salary range
        IF p_salary NOT BETWEEN c_min_salary AND c_max_salary THEN
            RAISE_APPLICATION_ERROR(-20011,
                'Salary $' || p_salary || ' is outside the allowed range ($' ||
                c_min_salary || ' - $' || c_max_salary || ')');
        END IF;

        -- Get a new employee ID from the sequence
        v_new_emp_id := employee_seq.NEXTVAL;

        -- Insert the employee
        INSERT INTO employees (employee_id, first_name, last_name, email,
                               hire_date, job_title, salary, department)
        VALUES (v_new_emp_id, p_first_name, p_last_name, p_email,
                SYSDATE, p_job_title, p_salary, p_department);

        COMMIT;
    END hire_employee;
END emp_mgmt_pkg;
```

```

        DBMS_OUTPUT.PUT_LINE('SUCCESS: Employee hired - ID: ' || v_new_emp_id);

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        log_error('hire_employee', 'Duplicate employee ID ' || v_new_emp_id);
        RAISE_APPLICATION_ERROR(-20012,
            'Employee ID ' || v_new_emp_id || ' already exists; check employee_seq');
    WHEN OTHERS THEN
        log_error('hire_employee', SQLERRM);
        ROLLBACK;
        RAISE; -- keep the original error (including -20010 and -20011)
END hire_employee;

PROCEDURE fire_employee(p_emp_id IN NUMBER) IS
    v_emp_name VARCHAR2(100);

    -- Raised when other employees still report to this one
    e_has_subordinates EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_has_subordinates, -2292);
BEGIN
    -- Check that the employee exists and get the name
    SELECT first_name || ' ' || last_name INTO v_emp_name
    FROM employees
    WHERE employee_id = p_emp_id;

    -- Delete the employee
    DELETE FROM employees WHERE employee_id = p_emp_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('SUCCESS: Employee ' || v_emp_name || ' removed');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20020, 'Employee ID ' || p_emp_id || ' not found');
    WHEN e_has_subordinates THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20021,
            v_emp_name || ' manages other staff; reassign them first');
    WHEN OTHERS THEN
        log_error('fire_employee', SQLERRM);
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20099,
            'Unexpected error in fire_employee: ' || SQLERRM);
END fire_employee;

PROCEDURE give_raise(p_emp_id IN NUMBER, p_percent IN NUMBER) IS
    v_old_salary NUMBER;
    v_new_salary NUMBER;
    v_emp_name VARCHAR2(100);
BEGIN
    -- Validate the raise percentage
    IF p_percent <= 0 OR p_percent > 50 THEN
        RAISE_APPLICATION_ERROR(-20030,
            'Invalid raise percentage: ' || p_percent ||
            '%. Must be between 0.1 and 50.');
```

```

        DBMS_OUTPUT.PUT_LINE('SUCCESS: ' || v_emp_name || ' salary: $' ||
                               v_old_salary || ' -> $' || ROUND(v_new_salary, 2));
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20031, 'Employee ID ' || p_emp_id || ' not found');
    WHEN OTHERS THEN
        log_error('give_raise', SQLERRM);
        ROLLBACK;
        RAISE; -- keep the original error (including -20030)
END give_raise;

FUNCTION get_employee_info(p_emp_id IN NUMBER) RETURN VARCHAR2 IS
    v_info VARCHAR2(500);
BEGIN
    SELECT first_name || ' ' || last_name ||
           ' - Job: ' || job_title ||
           ' - Dept: ' || department ||
           ' - Salary: $' || salary ||
           ' - Hired: ' || TO_CHAR(hire_date, 'DD-MON-YYYY')
    INTO v_info
    FROM employees
    WHERE employee_id = p_emp_id;

    RETURN v_info;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Employee ID ' || p_emp_id || ' not found';
    WHEN OTHERS THEN
        log_error('get_employee_info', SQLERRM);
        RETURN 'Error retrieving employee information';
END get_employee_info;

END emp_mgmt_pkg;
/

-- Test the package
BEGIN
    -- Valid operations
    DBMS_OUTPUT.PUT_LINE(emp_mgmt_pkg.get_employee_info(1));
    emp_mgmt_pkg.give_raise(3, 10);

    -- Employee 1 manages several staff members, so this is refused
    emp_mgmt_pkg.fire_employee(1);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Main program caught: ' || SQLERRM);
END;
/

```

Note: give_raise and hire_employee end their WHEN OTHERS handlers with a plain RAISE. That way a validation error such as -20030 reaches the caller unchanged instead of being replaced by a general "unexpected error" code. fire_employee shows the other approach: a named exception (ORA-02292, via PRAGMA EXCEPTION_INIT) turned into a clear business message.

6.11 Exception Handling Guidelines

1. Be specific with exception handling:

- ✓ Handle specific exceptions first and WHEN OTHERS last.
- ✓ Use meaningful error messages.
- ✓ Don't hide errors with empty WHEN OTHERS blocks.

2. Always clean up resources:

- ✓ ROLLBACK transactions on errors.

- ✓ Close cursors in exception handlers.
- ✓ Release locks and other resources.

3. Provide helpful information:

- ✓ Include context (which operation was being performed).
- ✓ Include the relevant data values.
- ✓ Use SQLCODE and SQLERRM for debugging.

4. Log errors appropriately:

- Log unexpected errors for investigation.
- Include timestamps and user information.
- Don't log expected business exceptions.

5. Use custom error codes consistently:

- ✓ Use the -20000 to -20999 range.
- ✓ Document your error codes.
- ✓ Group related errors (for example, -20100 to -20199 for enrollment errors).

Common Exception Handling Mistakes

Empty WHEN OTHERS:

```
EXCEPTION
  WHEN OTHERS THEN
    NULL;    -- BAD! Hides all errors
```

Not rolling back:

```
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error');
    -- Missing ROLLBACK; leaves partial changes behind
```

Generic error messages:

```
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error');    -- Not helpful!
```

Not handling expected exceptions:

```
-- If you expect NO_DATA_FOUND, handle it specifically
SELECT salary INTO v_sal FROM employees WHERE ...
-- Don't let it fall through to WHEN OTHERS
```

6.12 Real-World Exception Handling Patterns

Example 9: Batch Processing with Exception Handling

```
CREATE OR REPLACE PROCEDURE process_annual_bonuses IS
  -- Counters for reporting
  v_total_processed NUMBER := 0;
  v_success_count   NUMBER := 0;
  v_error_count     NUMBER := 0;

  CURSOR emp_cursor IS
    SELECT employee_id, first_name, last_name, salary, department
    FROM employees
    WHERE salary IS NOT NULL;
BEGIN
```

```

DBMS_OUTPUT.PUT_LINE('Starting annual bonus processing...');
DBMS_OUTPUT.PUT_LINE('=====');

FOR emp_rec IN emp_cursor LOOP
    v_total_processed := v_total_processed + 1;

    -- Process each employee in its own block
    DECLARE
        v_bonus          NUMBER;
        v_performance_rating NUMBER;
    BEGIN
        -- Performance rating (simulated; would normally come from a table)
        v_performance_rating := ROUND(DBMS_RANDOM.VALUE(1, 5));

        -- Calculate bonus (5% per rating point)
        v_bonus := emp_rec.salary * (v_performance_rating * 0.05);

        -- Business rule: minimum $1,000, maximum $8,000
        IF v_bonus < 1000 THEN
            v_bonus := 1000;
        ELSIF v_bonus > 8000 THEN
            v_bonus := 8000;
        END IF;

        -- Update the employee
        UPDATE employees
        SET salary = salary + v_bonus
        WHERE employee_id = emp_rec.employee_id;

        v_success_count := v_success_count + 1;

        -- Report progress every 10 employees
        IF MOD(v_success_count, 10) = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Processed ' || v_success_count || ' employees...');
        END IF;
    EXCEPTION
        WHEN OTHERS THEN
            -- Error for one employee: log it and continue with the others
            v_error_count := v_error_count + 1;
            DBMS_OUTPUT.PUT_LINE('ERROR processing employee ' ||
                                emp_rec.employee_id || ' (' ||
                                emp_rec.first_name || ' ' ||
                                emp_rec.last_name || '): ' || SQLERRM);
    END;
END LOOP;

-- Final commit for all successful updates
COMMIT;

-- Summary report
DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('Annual bonus processing completed');
DBMS_OUTPUT.PUT_LINE('Total employees: ' || v_total_processed);
DBMS_OUTPUT.PUT_LINE('Successfully processed: ' || v_success_count);
DBMS_OUTPUT.PUT_LINE('Errors: ' || v_error_count);

IF v_error_count > 0 THEN
    DBMS_OUTPUT.PUT_LINE('WARNING: ' || v_error_count || ' employees had errors');
END IF;
EXCEPTION
    WHEN OTHERS THEN
        -- Critical system error: roll back everything
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('CRITICAL ERROR: Bonus processing failed');
        DBMS_OUTPUT.PUT_LINE('All changes rolled back');
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        RAISE; -- Re-raise for the calling program
END process_annual_bonuses;
/

```

```

-- Execute the procedure
BEGIN
    process_annual_bonuses;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Main program: Bonus processing failed - ' || SQLERRM);
END;
/

```

Example 10: Exception Handling in Functions

```

CREATE OR REPLACE FUNCTION safe_calculate_tax
(
    p_salary    IN NUMBER,
    p_tax_type  IN VARCHAR2 DEFAULT 'STANDARD'
)
RETURN NUMBER
IS
    v_tax_rate  NUMBER;
    v_tax_amount NUMBER;

    -- Custom exceptions
    e_invalid_salary  EXCEPTION;
    e_invalid_tax_type EXCEPTION;
BEGIN
    -- Input validation
    IF p_salary IS NULL OR p_salary < 0 THEN
        RAISE e_invalid_salary;
    END IF;

    -- Determine the tax rate based on the type
    CASE UPPER(p_tax_type)
        WHEN 'STANDARD' THEN
            IF p_salary < 50000 THEN
                v_tax_rate := 0.10;
            ELSIF p_salary < 75000 THEN
                v_tax_rate := 0.15;
            ELSIF p_salary < 100000 THEN
                v_tax_rate := 0.20;
            ELSE
                v_tax_rate := 0.25;
            END IF;
        WHEN 'REDUCED' THEN
            v_tax_rate := 0.05;
        WHEN 'EXEMPT' THEN
            v_tax_rate := 0;
        ELSE
            RAISE e_invalid_tax_type;
    END CASE;

    v_tax_amount := p_salary * v_tax_rate;

    RETURN ROUND(v_tax_amount, 2);
EXCEPTION
    WHEN e_invalid_salary THEN
        RAISE_APPLICATION_ERROR(-20100,
            'Invalid salary amount: ' || NVL(TO_CHAR(p_salary), 'NULL') ||
            '. Salary must be a positive number. ');
    WHEN e_invalid_tax_type THEN
        RAISE_APPLICATION_ERROR(-20101,
            'Invalid tax type: ' || p_tax_type ||
            '. Valid types: STANDARD, REDUCED, EXEMPT. ');
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20199,
            'Unexpected error in tax calculation: ' || SQLERRM);
END safe_calculate_tax;
/

-- Test the function

```

```

DECLARE
    v_tax NUMBER;
BEGIN
    -- Valid cases
    v_tax := safe_calculate_tax(80000);
    DBMS_OUTPUT.PUT_LINE('Tax for $80000: $' || v_tax);

    v_tax := safe_calculate_tax(60000, 'REDUCED');
    DBMS_OUTPUT.PUT_LINE('Reduced tax for $60000: $' || v_tax);

    -- Invalid case
    v_tax := safe_calculate_tax(-1000);    -- Raises an exception
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

Summary of Chapter 6

1. Exception fundamentals: what exceptions are and why they matter, the basic exception handling syntax, and the propagation rules.
2. Types of exceptions: predefined Oracle exceptions (NO_DATA_FOUND and others), user-defined exceptions for business logic, and PRAGMA EXCEPTION_INIT for naming Oracle errors.
3. Exception information: the SQLCODE and SQLERRM functions, RAISE_APPLICATION_ERROR for custom messages, and re-raising exceptions with RAISE.
4. Good practice: specific handlers rather than generic ones, proper cleanup (ROLLBACK, closing cursors), meaningful error messages, and sensible error logging.

Quick Reference

```

-- Basic exception handling
BEGIN
    -- statements
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- handle a specific exception
    WHEN TOO_MANY_ROWS THEN
        -- handle another specific exception
    WHEN OTHERS THEN
        -- handle all other exceptions
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        ROLLBACK;
END;

-- User-defined exception
DECLARE
    e_custom_error EXCEPTION;
BEGIN
    IF condition THEN
        RAISE e_custom_error;
    END IF;
EXCEPTION
    WHEN e_custom_error THEN
        -- handle the custom exception
END;

-- PRAGMA EXCEPTION_INIT
DECLARE
    e_child_record_found EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_child_record_found, -2292);
BEGIN
    -- code that might cause ORA-02292

```

```

EXCEPTION
    WHEN e_child_record_found THEN
        -- handle it using a meaningful name
END;

-- RAISE_APPLICATION_ERROR
RAISE_APPLICATION_ERROR(-20001, 'Custom error message');

-- Re-raising an exception
EXCEPTION
    WHEN OTHERS THEN
        -- do some cleanup
        RAISE; -- re-raise the same exception

```

Common Predefined Exceptions

Exception	Oracle Error	Common Cause	Typical Handling
NO_DATA_FOUND	ORA-01403	SELECT INTO returns no rows	Check whether the record exists; provide a default
TOO_MANY_ROWS	ORA-01422	SELECT INTO returns several rows	Make the WHERE clause more specific
VALUE_ERROR	ORA-06502	Data conversion or size error	Validate input data; increase the variable size
DUP_VAL_ON_INDEX	ORA-00001	Unique constraint violation	Check for duplicates before INSERT
ZERO_DIVIDE	ORA-01476	Division by zero	Check for zero before dividing

Practice Exercises

1. Basic Exception Handling: Write a block that gets an employee's salary. Handle NO_DATA_FOUND with a meaningful message.
2. Multiple Exceptions: Create a procedure that updates an employee's salary. Handle NO_DATA_FOUND, VALUE_ERROR, and any other exceptions.
3. User-Defined Exception: Create a procedure that transfers an employee to another department (by department name). Create custom exceptions for an invalid target department, a target department that is the same as the current one, and employee not found.
4. PRAGMA EXCEPTION_INIT: Create a procedure that deletes a student. Use PRAGMA to handle the foreign key constraint error (ORA-02292) raised when the student still has enrollments, and give it a meaningful name.
5. RAISE_APPLICATION_ERROR: Create a function that calculates overtime pay. Use custom error codes (-20xxx) for negative hours, hours > 80, and an invalid employee ID.
6. Exception Propagation: Create nested blocks where the inner block raises an exception and the outer block handles it. Show both the handled and unhandled scenarios.
7. Re-raising Exceptions: Create a procedure that logs errors and then re-raises them. Call it from another block that handles the re-raised exception.
8. Batch Processing: Create a procedure that records scores for a list of enrollments. Handle errors for individual enrollments (for example a score above 100 or an unknown enrollment ID) without stopping the batch.
9. Function Exception Handling: Create a function that returns an employee's years of service. Handle all possible exceptions and return appropriate values or raise custom errors.

10. Complete Error Handling System: Create a package with constants for all error codes, a procedure for logging errors, and functions/procedures with thorough error handling. Test all the error scenarios.

Exception Handling Checklist

Before deploying code, check that:

- All expected exceptions are handled specifically
- WHEN OTHERS includes cleanup (ROLLBACK, closing cursors)
- Error messages are helpful to users
- Critical errors are logged for investigation
- Custom error codes are documented
- Exception handling doesn't hide important errors
- Resources are properly cleaned up in all scenarios

Chapter 7: Triggers: Automating Database Actions

7.1 Introduction to Triggers

What is a Trigger?

A trigger is a named PL/SQL block that is stored in the database and executes automatically when a specific event occurs on a table or view. Unlike procedures and functions, a trigger is never called manually. It fires on its own whenever the event it is watching happens.

Real-World Analogy

Think of a trigger like a smoke detector in a building:

- ✓ Without a trigger: someone has to stand in the room, watch for smoke, and call the fire brigade every time.
- ✓ With a trigger: the detector watches constantly. The moment smoke is detected, it sets off the alarm automatically, with no human involvement.

The trigger is the smoke detector, the event (INSERT, UPDATE, DELETE) is the smoke, and the action in the trigger body is the alarm.

Why Use Triggers?

Problem	Trigger Solution
Users forget to fill in a date	Fill it in automatically before the row is saved
Someone enters an invalid value	Block the operation and show an error message
You need to know who changed what data	Log the change automatically after it happens
Deleting a record should update another table	Cascade the change automatically
A business rule must always be enforced	Enforce it on every relevant operation

Trigger vs Stored Procedure vs Function

Feature	Trigger	Procedure	Function
Invocation	Automatic; fires on an event	Manual; called with EXEC or from a block	Manual; called in an assignment or in SQL
Parameters	None	Can have parameters	Can have parameters
Returns a value	No	No (uses OUT parameters)	Yes; mandatory
COMMIT / ROLLBACK	Not allowed inside the trigger	Allowed	Allowed
Used in SQL queries	No	No	Yes
Primary purpose	React to data events automatically	Perform an action	Compute and return a value

7.2 Types of Triggers

Oracle supports three main categories of triggers:

Category	Fires When	Common Use
DML triggers	An INSERT, UPDATE, or DELETE happens on a table or view	Validation, auditing, setting defaults, enforcing business rules (the most common type)
DDL triggers	A CREATE, ALTER, or DROP statement is executed	Logging changes to the database schema
System/database triggers	A database event such as LOGON, LOGOFF, or STARTUP occurs	Security auditing and session tracking

This chapter focuses on DML triggers, since they are by far the most commonly used in application development.

DML Trigger Sub-Types

Sub-Type	Description
BEFORE trigger	Fires before the change is saved to the table. You can still modify the data at this point.
AFTER trigger	Fires after the change has been made. Useful for logging and cascading.
Row-level trigger	Fires once for every row affected by the DML statement. Requires FOR EACH ROW.
Statement-level trigger	Fires once for the whole DML statement, however many rows are affected. No FOR EACH ROW.
INSTEAD OF trigger	Fires in place of the DML operation. Used on views that cannot be updated directly.

7.3 Trigger Syntax

Creating a Trigger

```

CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER}
{INSERT | UPDATE [OF column_name] | DELETE}
[OR {INSERT | UPDATE | DELETE}]
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
    -- Variable declarations (optional)
BEGIN
    -- Trigger body
EXCEPTION
    -- Exception handling (optional)
END [trigger_name];
/

```

Syntax Breakdown

Part	What It Means
CREATE OR REPLACE TRIGGER	Creates the trigger. If one with this name already exists, it is replaced without an error.

Part	What It Means
trigger_name	The name you give the trigger. Use descriptive names such as trg_validate_gpa or trg_audit_salary.
BEFORE AFTER	Whether the trigger fires before or after the change is saved.
INSERT UPDATE DELETE	The event that fires the trigger. You can combine several events using OR.
UPDATE OF column_name	Optional. Fires only when a specific column is updated, not any column.
ON table_name	The table the trigger is watching.
FOR EACH ROW	Makes the trigger row-level, so that it fires once per affected row. Without it, the trigger is statement-level.
WHEN (condition)	Optional filter. The body runs only if the condition is true. Row-level triggers only.
DECLARE ... BEGIN ... END	Standard PL/SQL block structure. DECLARE is optional if there are no local variables.

Naming Conventions

It is good practice to use a consistent prefix for trigger names so that they are easy to identify:

- ✓ the prefix trg_ followed by a descriptive name
- ✓ for example: trg_validate_gpa, trg_audit_salary, trg_set_hire_date

7.4 The :NEW and :OLD Pseudo-Records

When a row-level trigger fires, Oracle automatically provides two special records that give you access to the data before and after the change. These are called pseudo-records.

Pseudo-Record	What It Contains	Available In
:NEW	The new version of the row: the values being inserted, or the values the row is being updated to	INSERT, UPDATE
:OLD	The old version of the row: the values that existed before the change	UPDATE, DELETE

Availability by Event

Event	:NEW	:OLD
INSERT	Contains the new row being inserted	NULL; nothing existed before
UPDATE	Contains the new values after the change	Contains the old values before the change
DELETE	NULL; the row is being removed	Contains the row being deleted

Important Rules About :NEW and :OLD

- ✓ You can READ both :NEW and :OLD in any row-level trigger.
- ✓ You can MODIFY :NEW values only in a BEFORE trigger. In an AFTER trigger the row has already been changed, so it is too late.
- ✓ :NEW and :OLD are NOT available in statement-level triggers (triggers without FOR EACH ROW).
- ✓ You refer to individual columns with dot notation: :NEW.column_name or :OLD.column_name.
- ✓ In the WHEN clause the colon is left out: WHEN (NEW.salary > 10000).

7.5 BEFORE vs AFTER Triggers

Use BEFORE When	Use AFTER When
You want to modify :NEW values before the row is saved	You want to audit or log what was saved
You want to validate data and block an invalid operation	You want to update another related table after the change
You want to set default values automatically	You want to cascade a change to child records
You want to generate a primary key value automatically	You want to print a confirmation after the row is saved
You want to format data (uppercase, proper case, etc.)	You want to count or aggregate after the fact

The key rule: if you need to change what gets saved, use BEFORE. If you only need to react to what was saved, use AFTER.

7.6 Statement-Level vs Row-Level Triggers

	Statement-Level	Row-Level
Keyword required	No FOR EACH ROW clause	Must include FOR EACH ROW
How many times does it fire?	Exactly once per SQL statement, however many rows are affected	Once for every row affected by the statement
:NEW and :OLD available?	No; no access to individual row values	Yes; full access to :NEW and :OLD
Best use case	Logging that an event occurred, restricting DML at certain times	Validating each row, modifying data, auditing row by row
Example scenario	Log that someone ran an UPDATE on the table	Log what each individual row looked like before and after

If you run UPDATE students SET gpa = 4.0 WHERE status = 'Active' and it affects 50 rows, a statement-level trigger fires once and a row-level trigger fires 50 times.

7.7 Blocking Operations with RAISE_APPLICATION_ERROR

When you want a trigger to stop an INSERT, UPDATE, or DELETE from completing, use the built-in RAISE_APPLICATION_ERROR procedure (introduced in Chapter 6). It raises a custom error, and the statement that fired the trigger is cancelled.

Syntax

```
RAISE_APPLICATION_ERROR(error_number, 'Your error message here');
```

Part	Rule
error_number	Must be a negative number between -20000 and -20999. Oracle reserves this range for developers.
error message	A text string of up to 2048 characters. Write something meaningful so that the user understands what went wrong.

How the Error Appears to the User

When you write `RAISE_APPLICATION_ERROR(-20000, 'GPA must be between 0 and 4')`, Oracle displays:

```
ORA-20000: GPA must be between 0 and 4
```

Oracle converts the negative number to the `ORA-` format automatically. The operation is cancelled and nothing is saved.

For simplicity in class you can use `-20000` for all your custom errors. In a real production system you would assign different numbers to different categories of error to make debugging easier.

7.8 Trigger Predicates

When a trigger is defined to fire on several events (`INSERT OR UPDATE OR DELETE`), you need a way to know which event actually caused it to fire. Oracle provides three built-in Boolean predicates for this.

Predicate	Returns TRUE When	Returns FALSE When
INSERTING	The trigger fired because of an <code>INSERT</code> statement	The event was <code>UPDATE</code> or <code>DELETE</code>
UPDATING	The trigger fired because of an <code>UPDATE</code> statement	The event was <code>INSERT</code> or <code>DELETE</code>
DELETING	The trigger fired because of a <code>DELETE</code> statement	The event was <code>INSERT</code> or <code>UPDATE</code>

How to Use Predicates

```
IF INSERTING THEN
    -- logic for INSERT
ELSIF UPDATING THEN
    -- logic for UPDATE
ELSIF DELETING THEN
    -- logic for DELETE
END IF;
```

Using predicates lets you write one trigger instead of three separate ones, which is cleaner and easier to maintain.

UPDATING with a Specific Column

You can also check whether a specific column was updated:

```
IF UPDATING('SALARY') THEN
    -- runs only when the SALARY column was updated
END IF;
```

7.9 The Audit Trail Pattern

An audit trail is a separate table that records every change made to another table: who made the change, when, and what the values were before and after. Triggers are the standard way to implement audit trails in Oracle.

How It Works

- Create a separate audit table to store the history.
- Write an AFTER INSERT OR UPDATE OR DELETE trigger on the main table.
- Inside the trigger, INSERT a row into the audit table capturing the :OLD values, the :NEW values, USER, and SYSDATE.
- The audit table grows automatically; the developer never writes to it directly.

Built-in Values Useful for Auditing

Value	What It Returns
USER	The username of the person currently logged in and running the SQL
SYSDATE	The current date and time on the database server
SYSTIMESTAMP	The current date and time, including fractional seconds and time zone
SYS_CONTEXT('USERENV', 'IP_ADDRESS')	The IP address of the client machine that made the change

7.10 Cascading Changes with AFTER Triggers

An AFTER trigger can automatically update related tables when a change happens on the main table. This is called cascading: one change sets off a chain of related changes.

Common Cascading Scenarios

- ✓ When a student is set to Inactive, automatically deactivate all of their enrollments.
- ✓ When an employee is deleted, automatically delete their related records in other tables.
- ✓ When a department budget changes, automatically update a summary table.

SQL%ROWCOUNT

After a DML statement inside a trigger, SQL%ROWCOUNT tells you how many rows that statement affected. This is useful for building informative messages.

```

UPDATE enrollments
SET status = 'Inactive'
WHERE student_id = :OLD.student_id;

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' enrollment(s) deactivated.');
```

7.11 Viewing and Managing Triggers

Viewing Triggers in the Database

```

-- See all triggers on a specific table
SELECT trigger_name, trigger_type, triggering_event, status
FROM user_triggers
WHERE table_name = 'STUDENTS';

-- View the source code of a trigger
SELECT text
FROM user_source
WHERE name = 'TRG_VALIDATE_GPA'
ORDER BY line;

-- Check for compilation errors
SELECT line, position, text
FROM user_errors
WHERE name = 'TRG_VALIDATE_GPA'
ORDER BY line, position;
```

Managing Triggers

```

-- Disable a trigger (it still exists but stops firing)
ALTER TRIGGER trigger_name DISABLE;

-- Enable a disabled trigger
ALTER TRIGGER trigger_name ENABLE;

-- Disable ALL triggers on a table at once
ALTER TABLE table_name DISABLE ALL TRIGGERS;

-- Enable ALL triggers on a table at once
ALTER TABLE table_name ENABLE ALL TRIGGERS;

-- Remove a trigger permanently
DROP TRIGGER trigger_name;
```

7.12 Rules and Restrictions

Rule	Explanation
No COMMIT or ROLLBACK inside a trigger	A trigger runs as part of the calling transaction. Committing inside the trigger would commit the whole transaction unexpectedly.
Cannot call procedures that COMMIT or ROLLBACK	For the same reason, any COMMIT in the call chain is forbidden from inside a trigger.
A row-level trigger cannot query or modify its own table (mutating table)	If a row-level trigger on TABLE_A queries or updates TABLE_A, Oracle raises ORA-04091: table is mutating.

Rule	Explanation
No parameters	Triggers are fired by the database engine, so there is no caller to pass parameters.
:NEW can only be modified in BEFORE triggers	In AFTER triggers the row has already been changed. You can read :NEW but not change it.
:NEW and :OLD only in row-level triggers	Statement-level triggers have no access to individual row data.
Keep the number of triggers per table small	Oracle does not impose a practical limit, but many triggers on one table make its behaviour hard to follow and slow down DML.

Summary of Chapter 7

1. Trigger basics: what triggers are and why they fire automatically; how triggers differ from procedures and functions; the three categories (DML, DDL, and system triggers).
2. DML trigger structure: BEFORE vs AFTER timing and when to use each; row-level (FOR EACH ROW) vs statement-level triggers; the full syntax and keyword breakdown.
3. Working with row data: the :NEW and :OLD pseudo-records, which one is available for each event type, and where you can modify :NEW values.
4. Controlling operations: RAISE_APPLICATION_ERROR to block and cancel a DML operation, using custom error numbers in the -20000 to -20999 range.
5. Trigger predicates: INSERTING, UPDATING, and DELETING, which let one trigger handle several events cleanly.
6. Common patterns: setting default values and formatting data automatically, enforcing business rules and validating data, building a full audit trail with a separate audit table, and cascading changes to related tables with AFTER triggers.
7. Managing triggers: viewing them with user_triggers and user_source; disabling, enabling, and dropping them; understanding the restrictions and why they exist.

Quick Reference

```
-- Full trigger syntax
CREATE OR REPLACE TRIGGER trigger_name
{BEFORE | AFTER}
{INSERT | UPDATE [OF column] | DELETE} [OR ...]
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
    v_local_var datatype;
BEGIN
    -- Access row values:
    --     :NEW.column_name    (new value being saved)
    --     :OLD.column_name    (value before the change)

    -- Detect which event fired the trigger:
    IF INSERTING THEN ... END IF;
    IF UPDATING  THEN ... END IF;
    IF DELETING  THEN ... END IF;

    -- Block an operation:
    RAISE_APPLICATION_ERROR(-20000, 'Your message');
EXCEPTION
    WHEN exception THEN ...
END trigger_name;
```

/

```
-- Manage triggers
ALTER TRIGGER trigger_name DISABLE;
ALTER TRIGGER trigger_name ENABLE;
ALTER TABLE table_name DISABLE ALL TRIGGERS;
DROP TRIGGER trigger_name;
```

BEFORE vs AFTER Decision Guide

Choose BEFORE When	Choose AFTER When
Setting default values automatically	Logging or auditing the change
Validating and blocking bad data	Updating another related table
Formatting data before it is saved	Cascading a status change or deletion to child records
Generating a primary key automatically	Printing a confirmation after the row is saved

Common Errors and Solutions

Error	Cause	Solution
ORA-04091: table is mutating	A row-level trigger queries or updates the same table it is watching	Move the logic to a statement-level trigger, a compound trigger, or a stored procedure, or restructure it to avoid querying the same table
ORA-04084: cannot change NEW values for this trigger type	Assigning a value to :NEW in an AFTER trigger	Move all :NEW modifications to a BEFORE trigger
Trigger compiles but never fires	Wrong timing, event, or table name in the trigger definition	Check user_triggers and verify trigger_name, table_name, status, and triggering_event
ORA-04092: cannot COMMIT in a trigger	COMMIT or ROLLBACK written inside the trigger body	Remove COMMIT and ROLLBACK; the calling transaction handles them
ORA-00036: maximum number of recursive SQL levels exceeded	A trigger on table A updates table B, whose trigger updates table A again	Avoid circular trigger chains; redesign the logic to break the cycle
Trigger fires but :NEW or :OLD cannot be used	The trigger was written without FOR EACH ROW (statement-level)	Add FOR EACH ROW to make it a row-level trigger